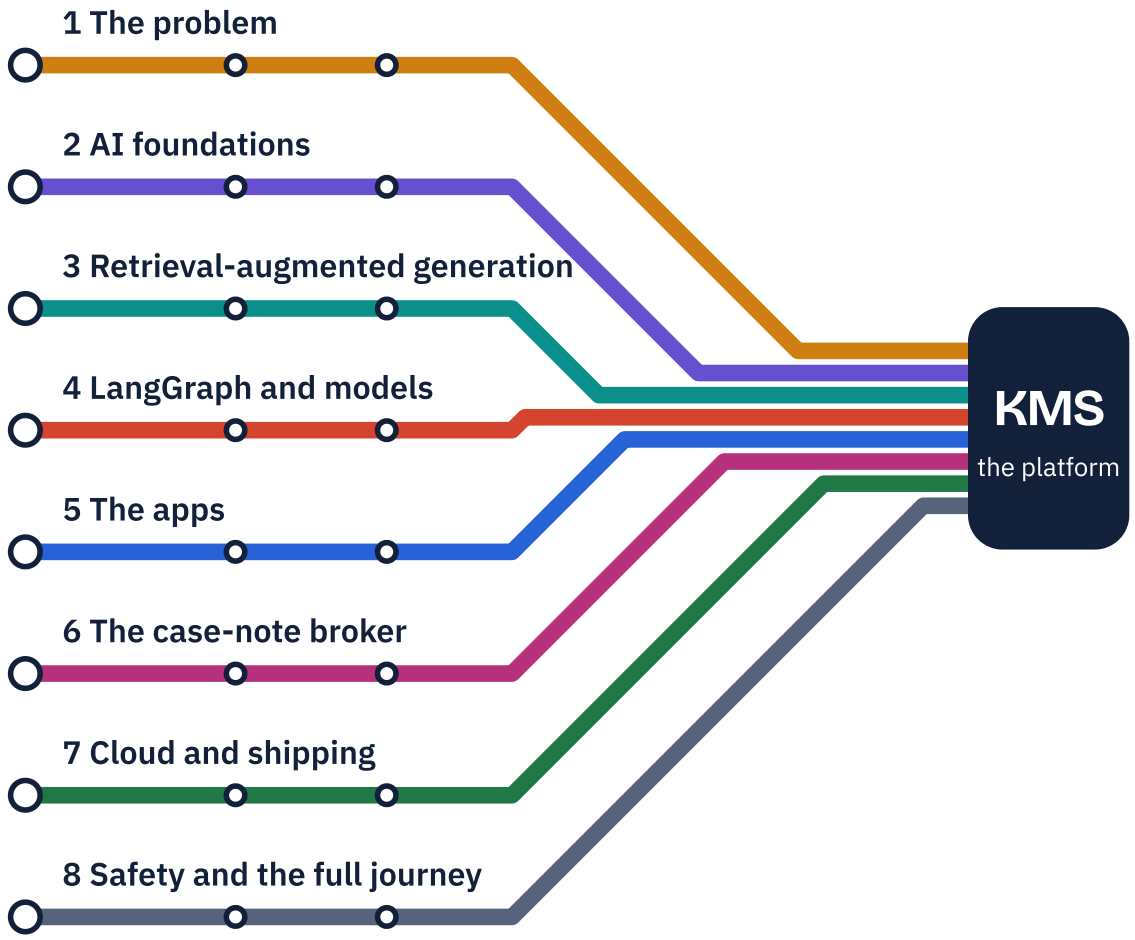
A FIELD GUIDE FOR NEWCOMERS

KMS, end to end

How a healthcare knowledge platform finds community resources with AI, connects to the mMR app, and ships to the cloud.

No AI or cloud background needed | 9 parts | 65 slides



How to read this guide

Every part of the guide is a coloured line, and every slide is a station on it. The badge in the top corner tells you which line you are on; the track along the bottom shows how far along it you are.

- **The problem** · why KMS exists and who uses it
- **AI foundations** · LLMs, tokens, embeddings, similarity
- **Retrieval-augmented generation** · grounding answers in real records
- **LangGraph and models** · workflows that loop, and the model gateway
- **The apps** · FastAPI, the mmr-api proxy, Streamlit
- **The case-note broker** · async jobs, webhooks, signatures
- **Cloud and shipping** · Docker, AWS, Terraform, CI/CD
- **Safety and the full journey** · healthcare rules, one request traced end to end

Three panels recur throughout

> **IN PLAIN WORDS**
> An everyday comparison for a technical idea. Read these first when a slide feels dense.

IN THE CODE `backend/utils/search_graph.py`

Where the idea lives in the real code, styled like a station sign.

Q1 What does RAG stand for?

Questions open with a click. Use ← → to move between slides. The guide prints one slide per page.

Contents

1 **The problem**
Slides 4–7 · Why KMS exists · who uses it · the whole system on one map

2 **AI foundations**
Slides 8–13 · LLMs · tokens · hallucination · embeddings · similarity

3 **Retrieval-augmented generation**
Slides 14–23 · RAG · chunking · contextual headers · vector index · hybrid search

4 **LangGraph and models**
Slides 24–30 · LangChain · LangGraph · the two graphs · LiteLLM

5 **The apps**
Slides 31–40 · APIs · FastAPI · JWT · the mmr-api proxy · Streamlit

6 **The case-note broker**
Slides 41–46 · DAP notes · async · webhooks · HMAC · expiring keys

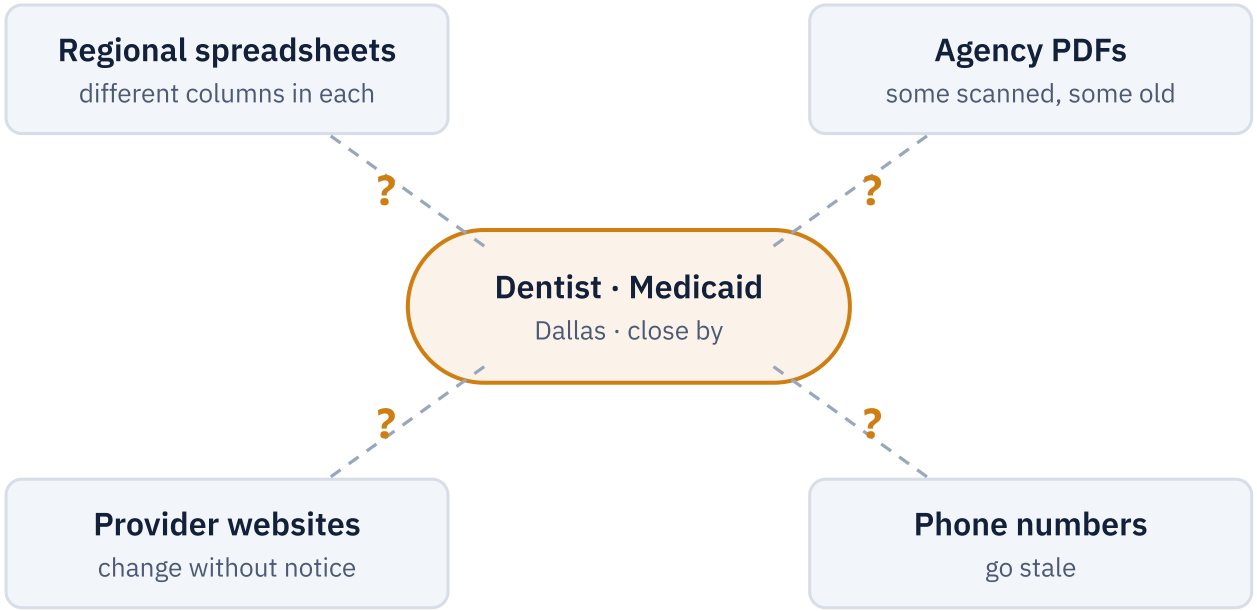
7 **Cloud and shipping**
Slides 47–55 · Data stores · Docker · AWS · Terraform · CI/CD · rollouts

8 **Safety and the full journey**
Slides 56–58 · Healthcare principles · one question end to end

9 **Practice and reference**
Slides 59–65 · Quizzes · exercises · reading order · glossary

The question behind it all

A care navigator's member needs a dentist: on Medicaid, in Dallas, can't travel far. Some list somewhere answers this. Which one, and is it still right?



Scattered

Dozens of directories, each with its own columns: “parish” in one file, “county” in another.

Stale

Clinics move and numbers change. A wrong number is a referral that fails.

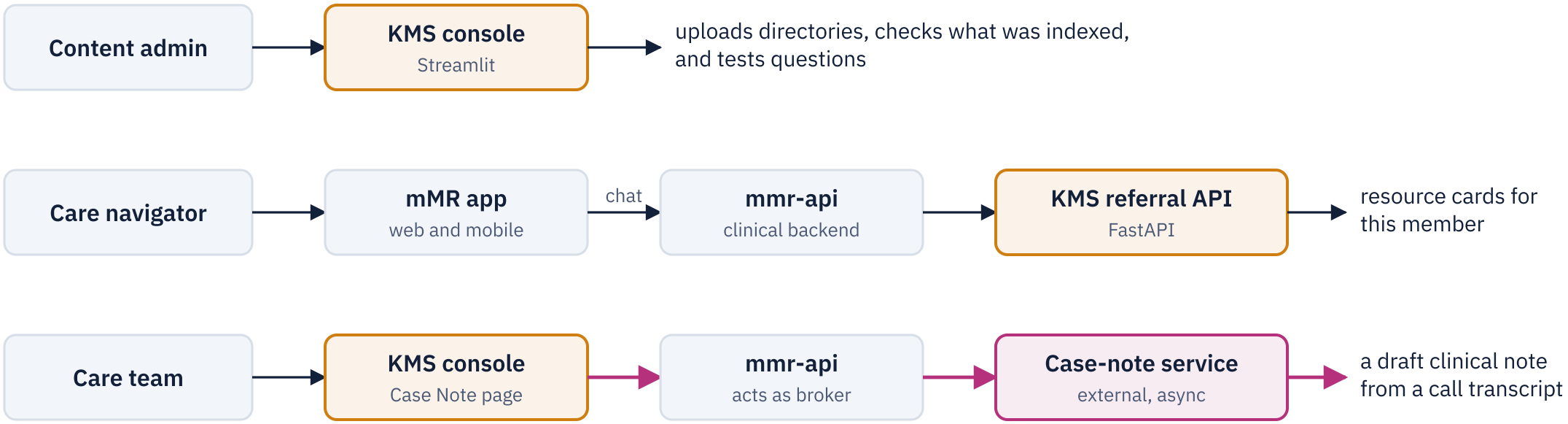
Literal

A keyword search for “tooth pain” never finds “dental clinic”.

IN PLAIN WORDS

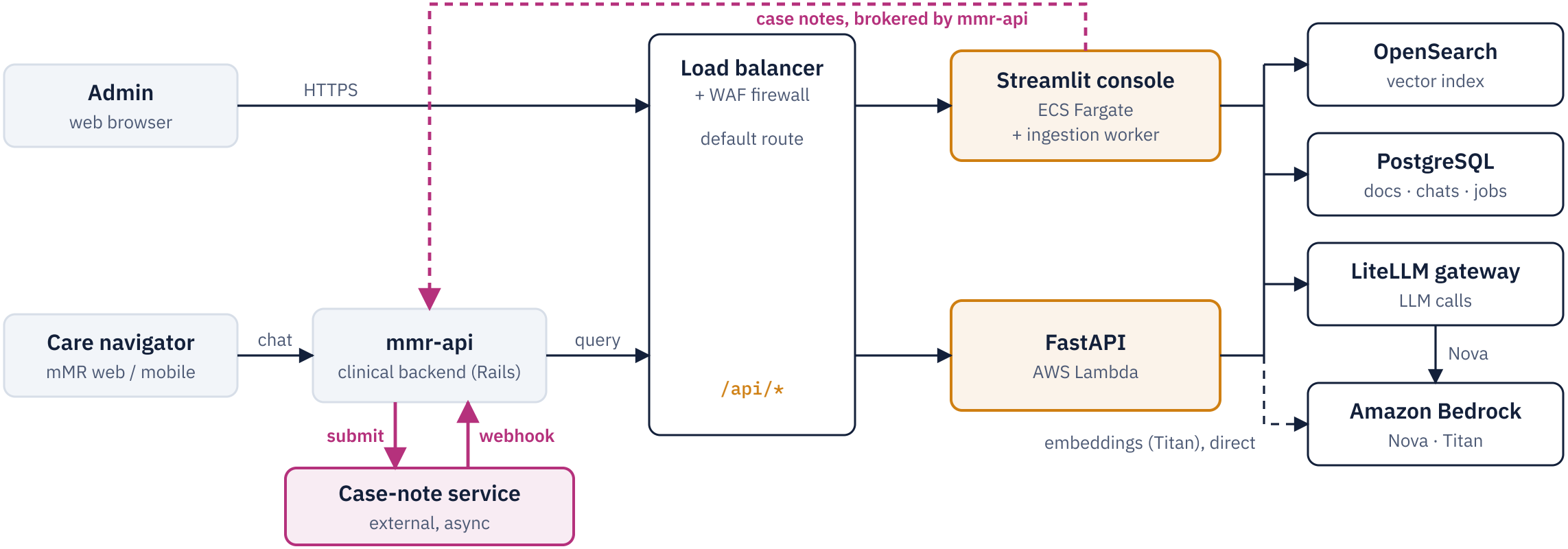
KMS turns a pile of directories into something you can ask in plain language, and answers with real records rather than guesses.

Who uses KMS: three ways in



Highlighted boxes are KMS itself. Two of the three paths pass through **mmr-api**, the clinical platform's main backend. That name comes back in parts 5 and 6.

The whole system on one map



One load balancer serves both front doors: the admin console, and the API that mmr-api calls for the mMR app. Both apps read the same index and reach models through one gateway. The magenta path is the case-note flow, which runs through mmr-api because that is the only address the note service calls back.

Three runtimes, one codebase

ECS FARGATE · ALWAYS ON

Streamlit console

The admin web app: upload directories, inspect the index, test searches, generate case notes. Port 8501, behind the load balancer.

`backend/Home.py + pages/`

SAME CONTAINER AS THE CONSOLE

Ingestion worker

A background process that picks up upload jobs from PostgreSQL, so a 400-row spreadsheet doesn't freeze the page.

`backend/utils/ingestion_worker.py`

AWS LAMBDA · RUNS ON DEMAND

FastAPI service

The API mmr-api calls for referral cards and chat. Billed per request; pinged every five minutes to stay warm.

`backend/api/main.py`

SURPRISE 1

The worker hides inside the console's container

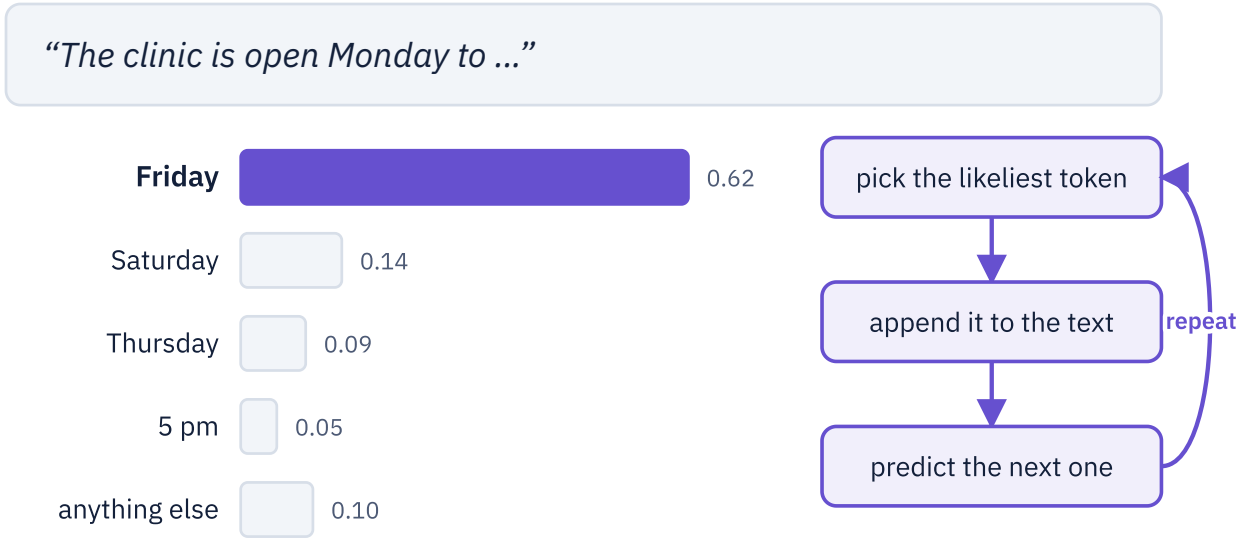
The Dockerfile's last line starts both: `python utils/ingestion_worker.py & streamlit run Home.py`. One deployment, two processes.

SURPRISE 2

The Lambda runs only part of the API

The admin and job routes need a PostgreSQL driver the slim Lambda image doesn't ship, so `api/main.py` imports them inside `try/except`. Add a database import to Lambda code and the whole referral API returns 502. That has happened once.

What a large language model actually does



Illustrative numbers. The model scores every possible next token, picks one, appends it and repeats until the answer is done.

An LLM is a next-word predictor trained on an enormous amount of text. Give it the start of something and it writes the most plausible continuation, one small piece at a time.

IN PLAIN WORDS

Autocomplete that has read most of the internet. It is fluent by design, and correct only when the right facts happen to be in play.

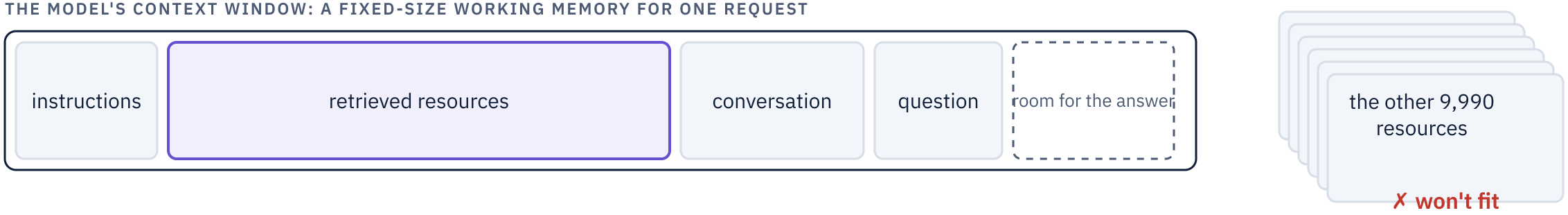
IN KMS

```
Amazon Nova Lite, reached through the LiteLLM gateway
```

Tokens and the context window

`Find` `·dental` `·clin` `ics` `·near` `·Dall` `as` `?` → **8 tokens**

THE MODEL'S CONTEXT WINDOW: A FIXED-SIZE WORKING MEMORY FOR ONE REQUEST



A token is a word-piece

Models read and write tokens, not words: roughly three quarters of an English word each, on average. Rare words split into several.

The window is the limit

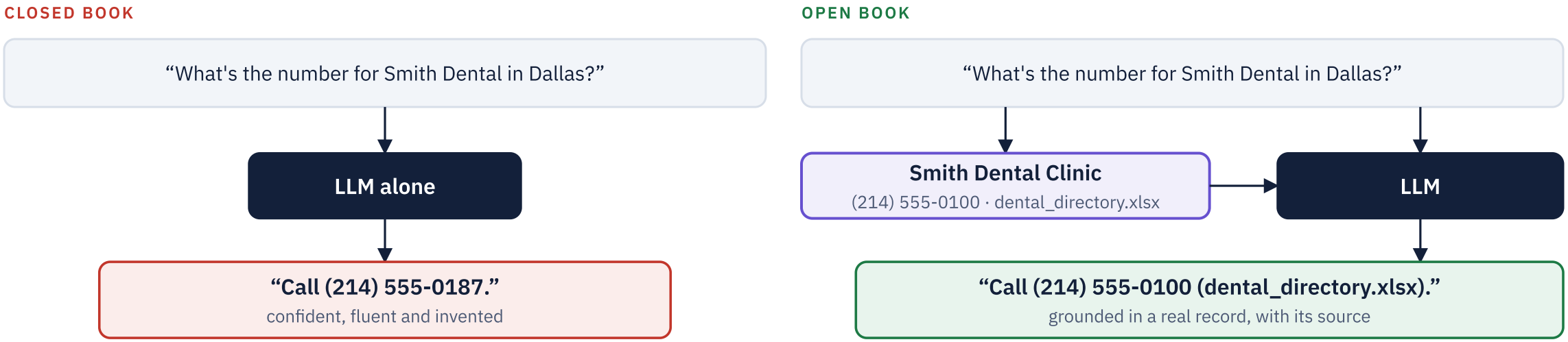
Everything the model should consider for one answer must fit inside it. You cannot paste a whole directory, so something has to choose what goes in.

You pay per token

Tokens in and tokens out are billed. KMS's own estimate is about $0.00004 per question, small because only the best matches are sent.

Why models make things up

When a model doesn't know something, it still writes something that sounds right. That's called a hallucination. In healthcare it isn't a typo: a wrong phone number is a member who doesn't reach care.



It has never seen your data

Your directories were never in its training text.

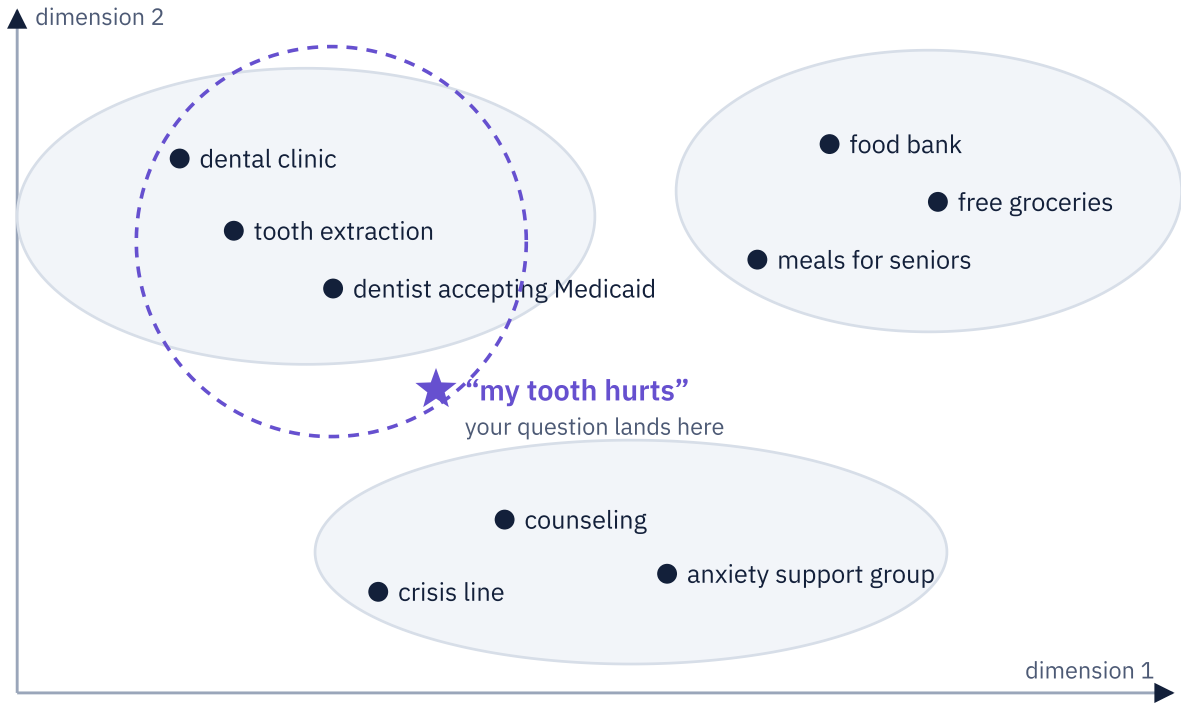
Its knowledge is frozen

It learned up to a cut-off date. Clinics have moved since.

It is trained to be helpful

Sounding confident scores better than “I don't know”. The fix is to hand it the facts.

Embeddings: meaning as coordinates



Drawn in two dimensions. Real embeddings use far more.

An embedding model turns any text into a long list of numbers, a point in space. It is trained so that texts with similar meaning land close together, even when they share no words.

IN KMS

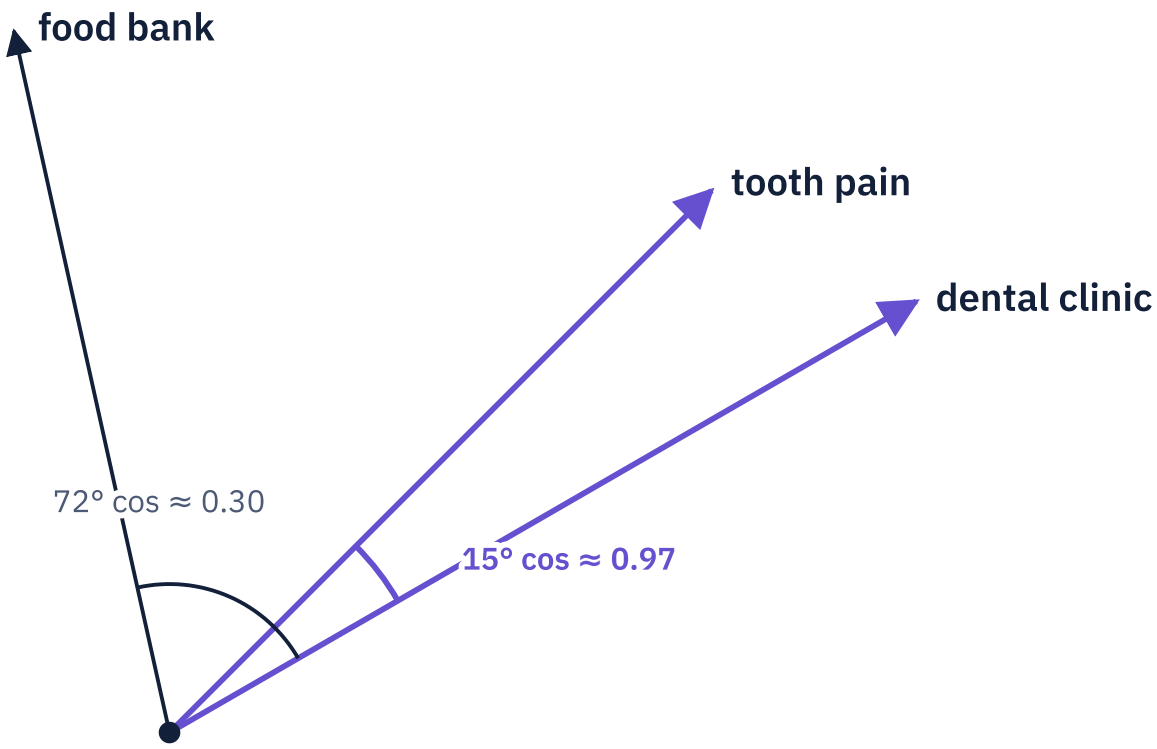
Amazon Titan Text Embeddings v2 gives every chunk and every question **1,024 numbers**.

IN PLAIN WORDS

A map where distance means difference in meaning. Your question becomes a pin; the answers are whatever sits nearby.

IN THE CODE `backend/src/core/embeddings/titan.py`

Measuring “close”: cosine similarity



Illustrative angles. Only direction matters, not length.

Each embedding is an arrow from the origin. Two texts are similar when their arrows point the same way, so the angle between them is the measure.

THE FORMULA

$$\cos\theta = (a \cdot b) \div (|a| \times |b|)$$

1 means the same direction. 0 means unrelated. Length is ignored, so a long description and a short one can still match.

IN PLAIN WORDS

Two arrows pointing the same way mean the same thing, however long they are.

OpenSearch does this comparison across every stored chunk; part 3 shows how it avoids checking them one by one.

Keyword search versus vector search

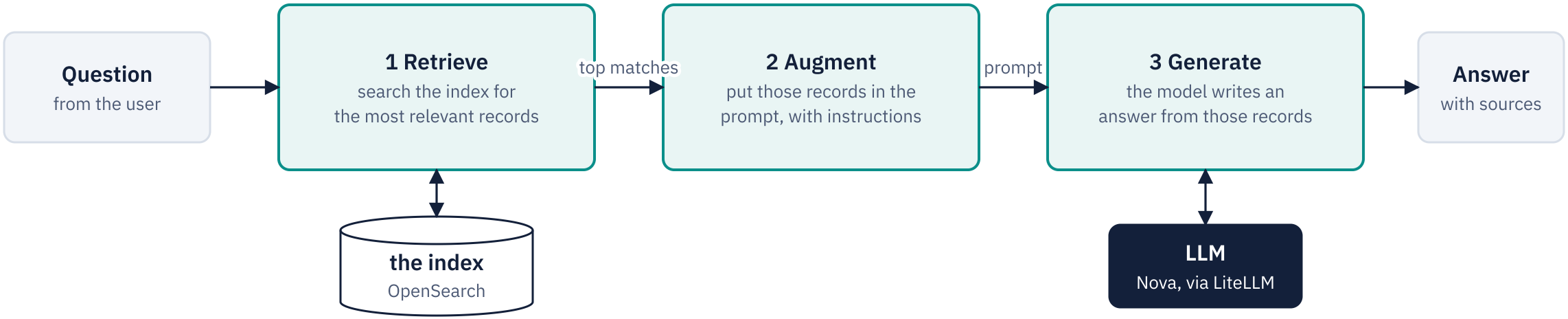
The question is “my tooth hurts”. Each kind of search is excellent at something and blind to something else.

	Keyword search	Vector search	Filters
Finds “dental clinic” for “tooth hurts”	✗ no shared words	✓ same meaning	–
Matches an exact name or ZIP code	✓ “Smith Dental”, “75201”	~ numbers have little meaning	–
Enforces “only in Dallas”	~ only if the word appears	✗ a Houston clinic is just as close	✓ a hard rule
Typical failure	misses synonyms	a perfect match in the wrong city	finds nothing by itself

SO

Each one covers another's blind spot, so KMS runs all three inside a single query. That combination is called **hybrid search**, and it's coming up in part 3.

RAG: let the model read before it answers



IN PLAIN WORDS

An open-book exam. The model isn't expected to remember your directories; it is handed the right pages moments before it answers, and told to stick to them.

Why RAG, rather than retraining the model?

	Just ask the model	Fine-tune the model	RAG
Knows your private directories	✗ never saw them	~ partly, baked in	✓ reads them live
Stays current when a number changes	✗	✗ retrain to update	✓ re-index one row
Shows where an answer came from	✗	✗	✓ a source on every card
Cost to update	none	high	low
Risk of invented facts	high	still present	lowest

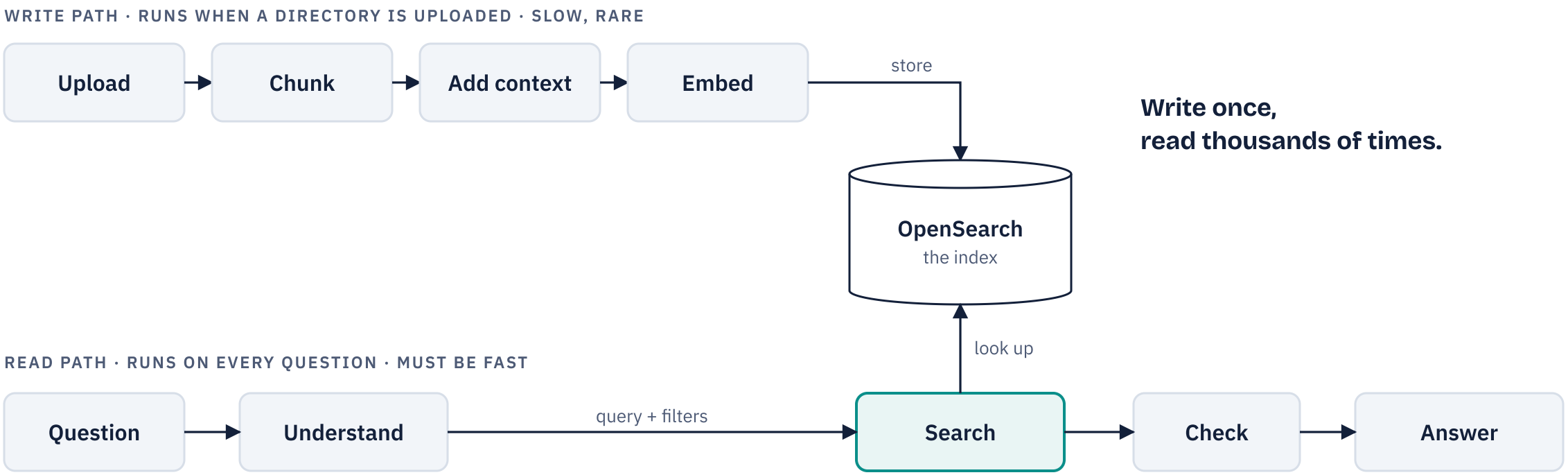
IN PLAIN WORDS

Fine-tuning teaches a model a skill or a style. RAG hands it the facts. A resource directory is all facts.

THE VERDICT FOR KMS

For directories that change every week, RAG wins on every axis that matters. Everything in the next nine slides is about doing it well.

Two halves: the write path and the read path



Write path · ingestion

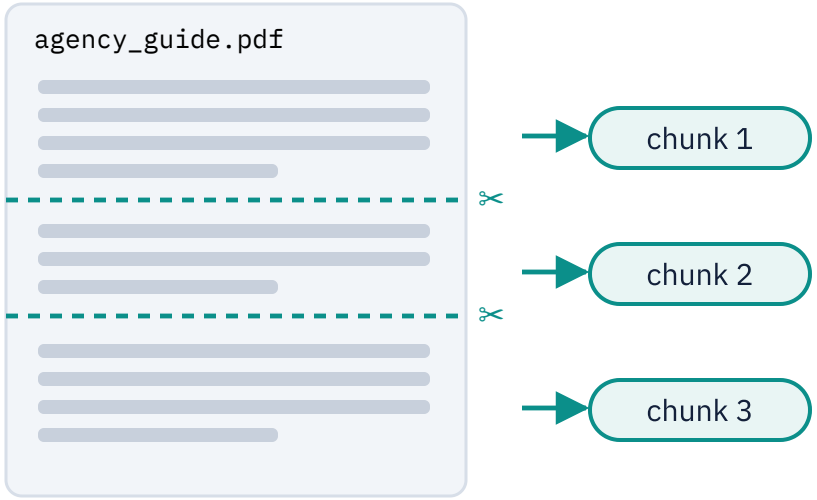
Slow and careful: an LLM call per chunk to write context, an embedding per chunk. Runs in the background worker so nobody waits.

Read path · retrieval

Fast and frequent: every chat message and every referral request. Its only job is to find the right chunks quickly and answer from them.

Chunking: cutting documents into findable pieces

A PDF · CUT BY PARAGRAPH OR PAGE



A SPREADSHEET · ONE ROW IS ONE RESOURCE

Name	Phone	City	Category	
Smith Dental	(214) 555-0100	Dallas	Dental	chunk · row 2
Elm St Pantry	(214) 555-0142	Dallas	Food	chunk · row 3
Oak Counseling	(972) 555-0199	Irving	Mental	chunk · row 4
Hope Senior Meals	(214) 555-0123	Dallas	Food	chunk · row 5

Why cut at all?

One embedding summarises a whole text as one point. Embed a 40-page PDF and every topic blurs into one vague point; small focused pieces stay precise.

How KMS cuts

Sentence boundaries, paragraph groups or pages for documents. For spreadsheets, one row per chunk, because a row *is* a resource.

Too big or too small

Too big and the point is blurry. Too small and it loses meaning: “555-0100” alone says nothing. The next slide fixes the second problem.

IN THE CODE `backend/utils/chunker.py`

Contextual retrieval: a label on every chunk

BEFORE · WHAT USED TO BE EMBEDDED

```
Smith Dental Clinic | (214) 555-0100 | 1200 Elm St | Dental
```

↓

AFTER · WHAT KMS EMBEDS

```
Dental care providers in Dallas County, Texas, from the
regional Medicaid resource directory, sheet "Dental".
Smith Dental Clinic | (214) 555-0100 | 1200 Elm St | Dental
```

IN PLAIN WORDS

A box labelled “spare phone chargers, kitchen drawer” is findable. The same chargers in an unlabelled box are not.

1. At upload, an LLM reads what the document is about: its sheets, columns and region.
2. It writes a one-sentence **context header** for each chunk.
3. The header is joined to the chunk *before* embedding, and also stored, so the answering model can read it too.

49% fewer failed retrievals in Anthropic's published benchmark for this technique, the figure the KMS docs cite.

IN THE CODE `backend/utils/contextual_chunker.py`

Where the vectors live: OpenSearch and HNSW



Comparing a question with every stored vector one by one is too slow. **HNSW** (hierarchical navigable small world) builds layered shortcuts instead.

Start on the sparse top layer, hop towards the question, drop a layer, repeat. You arrive near the closest matches having visited a handful of points.

IN PLAIN WORDS

Find an address by motorway to the right city, main roads to the district, then the street. You never knock on every door.

IN KMS

A `knn_vector` field of 1,024 dimensions, HNSW via nmslib, `m=16`, `ef_search=100`. Its distance setting is `space_type: cosinesimil`: the cosine similarity from part 2.

What one indexed resource looks like

```
{
  "resource_name":   "Smith Dental Clinic",
  "text":            "Smith Dental Clinic | general and emergency dentistry"
  "context_header":  "Dental care providers in Dallas County, Texas …",
  "embedding":       [0.021, -0.113, 0.067, …],   // 1,024 numbers
  "category_field":  "Dental",
  "city":            "Dallas",
  "county":          "Dallas",
  "state":           "TX",
  "zip_code":        "75201",
  "phone":           "(214) 555-0100",
  "website_url":     "https://example.org/smith-dental",
  "url_valid":       true,
  "url_extracted_phone": "(214) 555-0100",
  "filename":        "dental_directory.xlsx",
  "document_type":   "excel"
}
```

A synthetic example in the shape KMS stores. Every field has a job.

Meaning

Used by vector search to match what a question is about.

`embedding · context_header`

Words

Used by keyword search for exact names and phrases.

`resource_name · text`

Facts

Used as filters: hard rules no score can override.

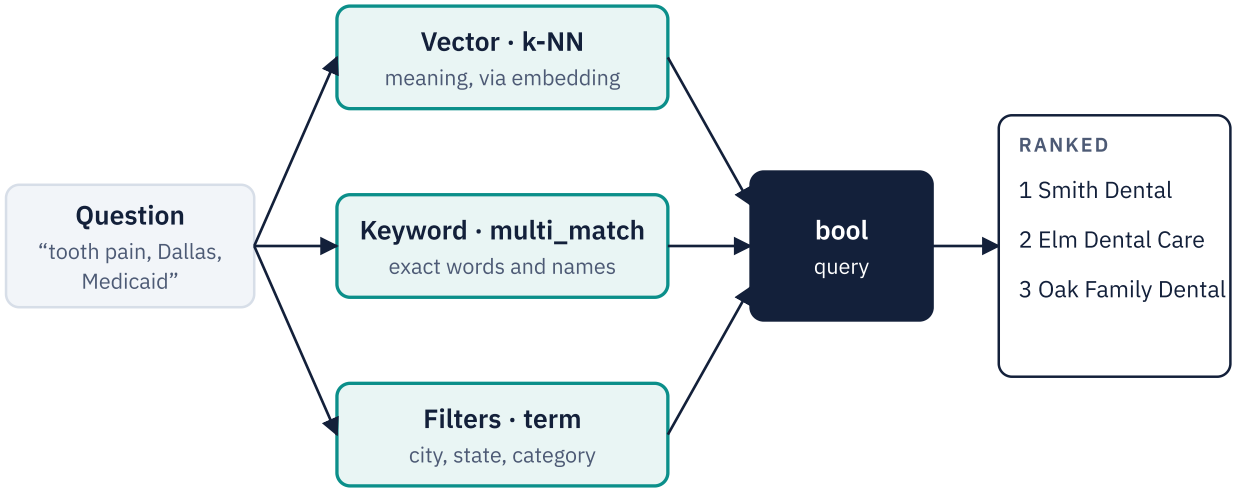
`category_field · city · county · state · zip_code`

Freshness

Filled by visiting the website, to catch numbers that went stale.

`url_valid · url_extracted_phone`

Hybrid search: three signals, one ranking



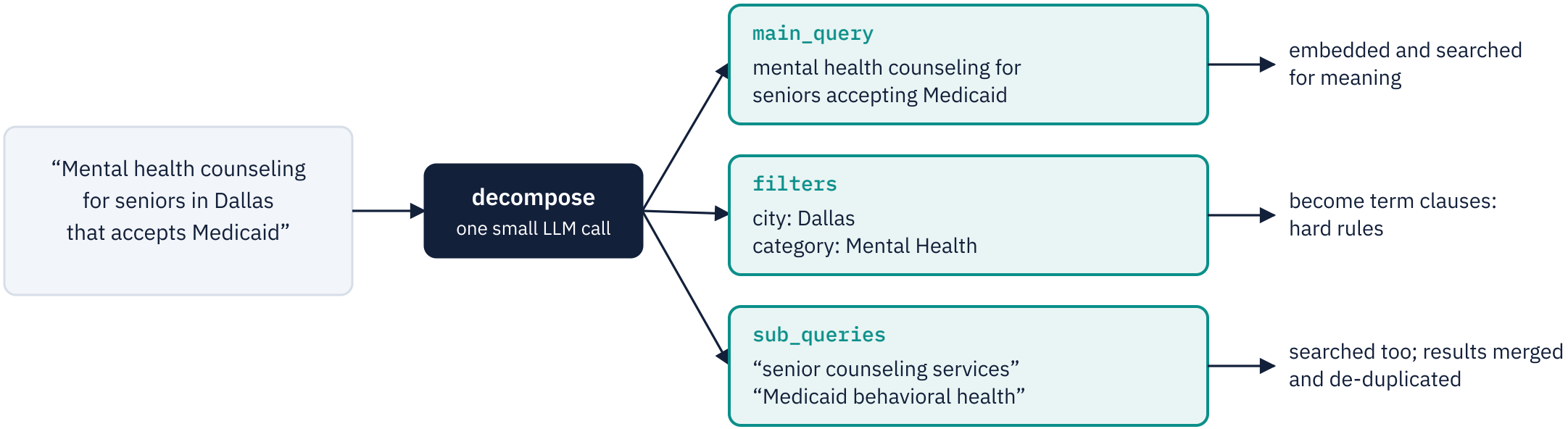

```
{ "query": { "bool": {
    "filter": [
      { "term": { "city": "dallas" } },
      { "term": { "category_field": "dental" } } ],
    "should": [
      { "knn": { "embedding": {
          "vector": [0.02, -0.11, …], "k": 10 } } },
      { "multi_match": {
          "query": "tooth pain",
          "fields": ["resource_name", "text"] } } ]
} } }
```

A simplified sketch of the idea, not the exact production query.

filter clauses are pass/fail rules. **should** clauses add to a result's score, so a match on both meaning and words rises to the top.

IN THE CODE `backend/utils/search_utils.py`

Query decomposition: understand the question first



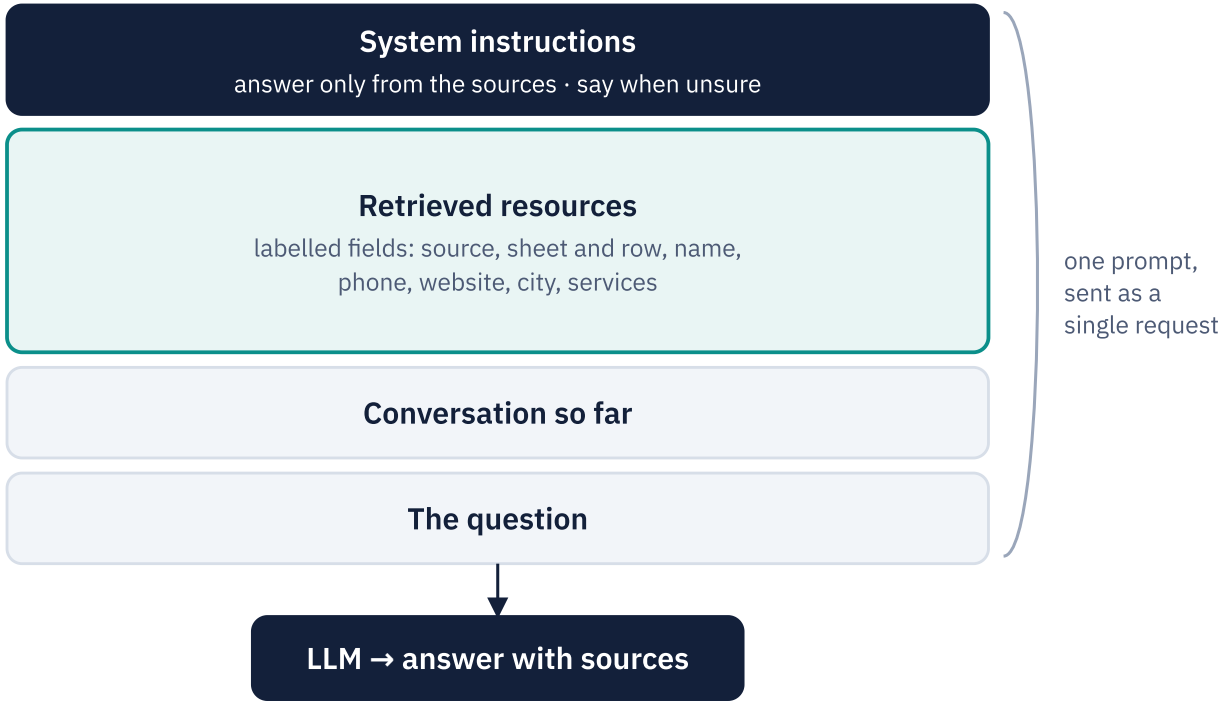
IN PLAIN WORDS

Before searching a library you work out what you're really after: the topic, the must-haves, and a couple of other ways to phrase it.

IN THE CODE `backend/utils/query_processor.py`

`decompose_query()` `build_opensearch_filters()`

Generation: the prompt sandwich




```
Source: dental_directory.xlsx
Sheet: Dental, Row 14
Resource: Smith Dental Clinic
Phone: (214) 555-0100
Website: https://example.org/smith-dental
City: Dallas, TX
Services: general and emergency dentistry
```

One retrieved resource as the model sees it. Synthetic data.

- Labelled fields let the model quote a phone number exactly instead of paraphrasing it.
- The source, sheet and row let every answer cite exactly where it came from.
- The instructions forbid filling gaps from memory.

Assembled in `generate_response()`.

From one call to a workflow: LangChain and LangGraph

A useful AI feature is rarely a single model call. KMS's search classifies the question, searches, judges the results, maybe rewrites and searches again, then answers. Two libraries organise that.

LANGCHAIN · THE PARTS BIN

Standard building blocks

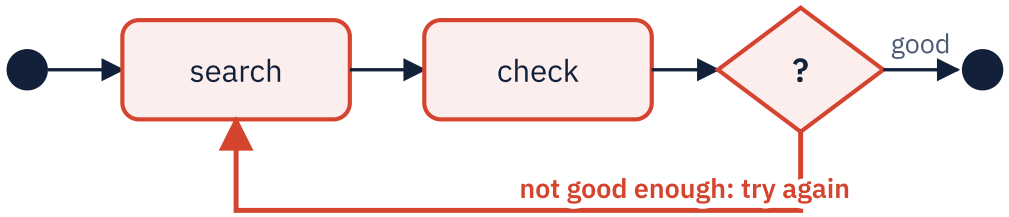
Ready-made pieces every AI app needs, with the same interface whichever model sits behind them.

- model wrappers
- prompt templates
- message types
- output parsers
- retrievers
- tool definitions

LANGGRAPH · THE WIRING

Steps as a graph that can loop

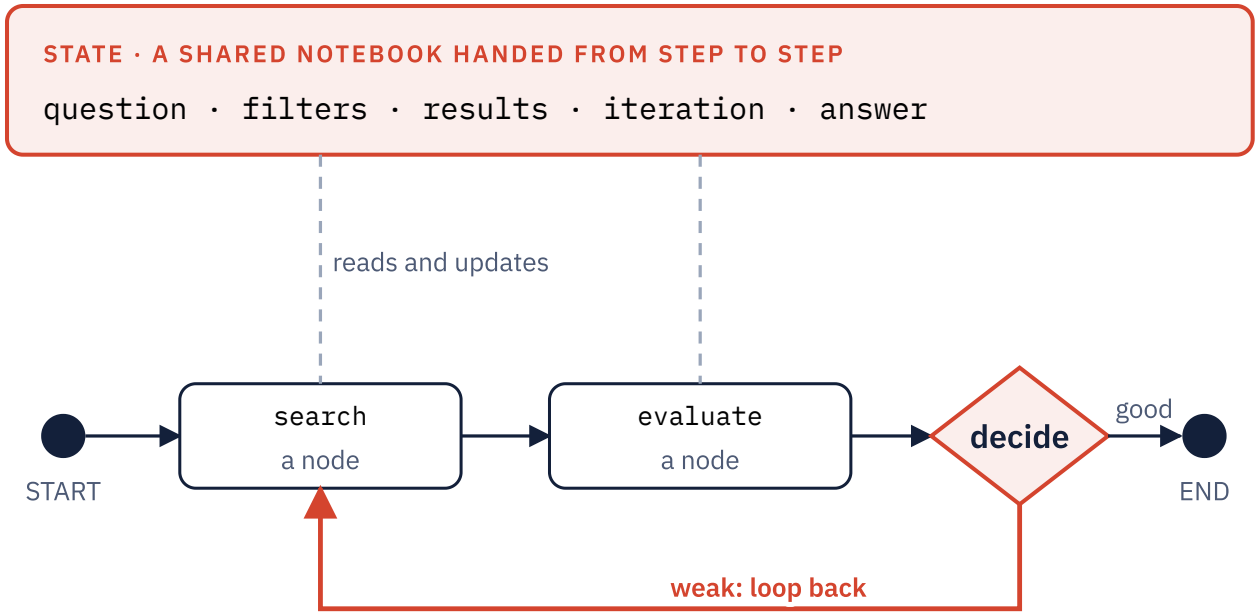
Arrange steps as nodes and arrows, with memory passed along. Unlike a straight chain, a graph can branch and go back.



IN PLAIN WORDS

LangChain is the box of LEGO bricks. LangGraph is the instruction booklet, the kind that can say “if this doesn't fit, go back to step 2”.

LangGraph's vocabulary in one picture



Node: a Python function. **Edge**: what runs next. **Conditional edge**: a function that picks the next node from the state.

```
graph = StateGraph(SearchState)
graph.add_node("search", search)
graph.add_node("evaluate", evaluate)

graph.add_edge(START, "search")
graph.add_edge("search", "evaluate")
graph.add_conditional_edges(
    "evaluate", decide,
    {"good": END, "weak": "search"})

app = graph.compile()
result = app.invoke({"question": "…"})
```

Simplified. KMS's real state type is `AgentState`.

Each node receives the state, does one job, and returns only the fields it changed. The graph merges them and decides what runs next.

The ingestion graph: six stations from file to index

INGESTION GRAPH · RUNS IN THE BACKGROUND WORKER · PROGRESS SAVED AS A JOB IN POSTGRESQL



Stable ids

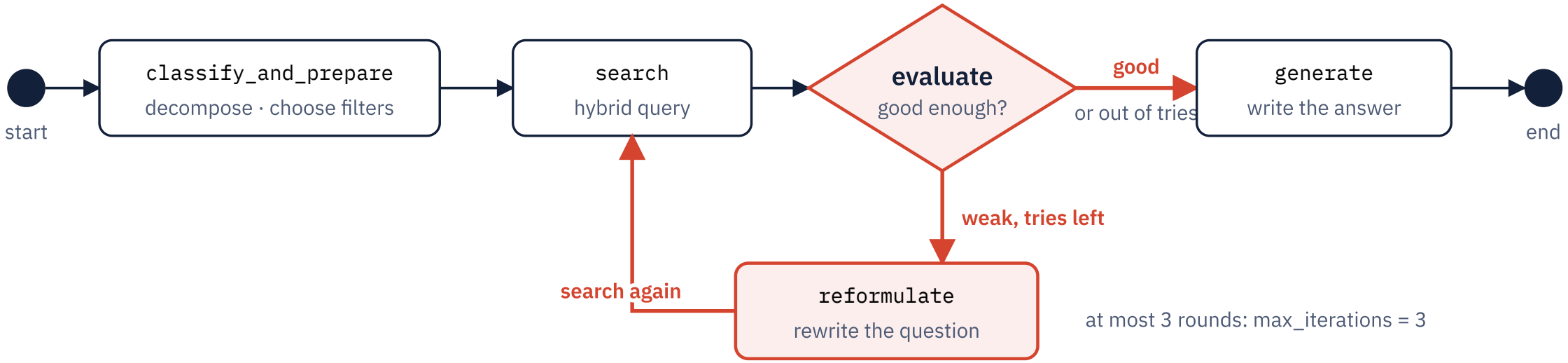
Every chunk is stored as `{{document_id}}_{{index}}`. Reprocessing a file updates records in place instead of duplicating them, and later URL checks can find the exact record to update.

Only what changed

Each spreadsheet row gets a fingerprint, kept in the `excel_row_hashes` table. Re-upload a directory and only the rows that changed are labelled and embedded again: less waiting, fewer paid model calls.

IN THE CODE `backend/utils/ingestion_graph.py` `backend/utils/column_mapper.py`

The search graph: an AI that checks its own work



IN PLAIN WORDS

A student who reads the question, looks things up, checks whether the notes actually answer it, and if not rephrases the search and tries again, before writing the essay.

IN THE CODE `backend/utils/search_graph.py`

```
GraphConfig(max_iterations=3)
```

The loop in action

ROUND 1

`tooth hurt help ASAP`

- City Pain Management Center
- Main Street Pharmacy

evaluate: weak · no dental providers

REFORMULATE

→

`emergency dental clinic, Dallas, accepts Medicaid`

ROUND 2

`emergency dental clinic …`

- Smith Dental Clinic
- Elm Dental Care

evaluate: good · generate the answer

What “agentic” means here

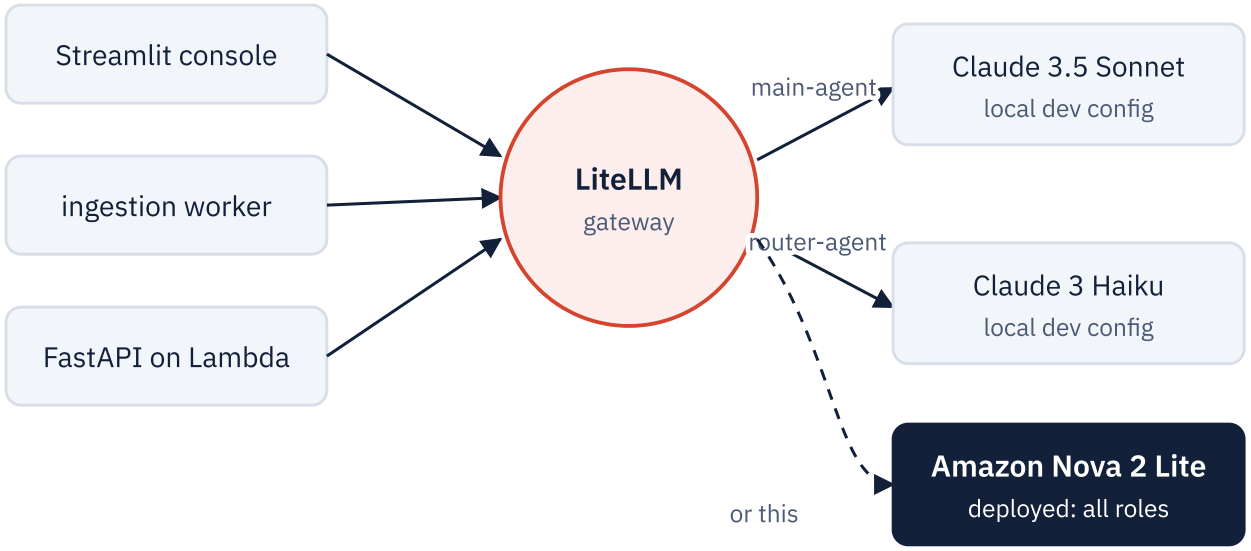
The pipeline makes a decision about its own next step. Classic RAG would have answered confidently from the pharmacy and the pain clinic.

A loop needs a brake

Three rounds at most, then answer with the best it has. Any system that can retry must also be told when to stop, or one odd question becomes an endless bill.

Illustrative example with synthetic resources.

LiteLLM: one socket for every model



KMS never calls a model vendor directly. It asks a gateway for a **role**, such as `main-agent`, and configuration decides which real model answers.

THE PROOF

The repo's local gateway maps those roles to Claude models on Bedrock. The deployed environment sets every role to Amazon Nova 2 Lite. Same code, different models, changed only in configuration.

- One request format and one key store for every provider
- Usage and cost tracked centrally
- Swap or fall back to another model without a deploy

IN PLAIN WORDS

A universal travel adapter: the app always plugs into the same socket.

Which model does which job, and what it costs

Job	Role	When it runs
Classify the question and plan filters	`router-agent` · small and fast	every question
Judge whether results are good enough	a small, cheap model	every search round
Write the final answer	`main-agent` · the most capable	every question
Write context headers at upload	an LLM via the gateway	once per chunk, at ingestion
Turn text into vectors	Amazon Titan Text Embeddings v2	every chunk at upload, every question

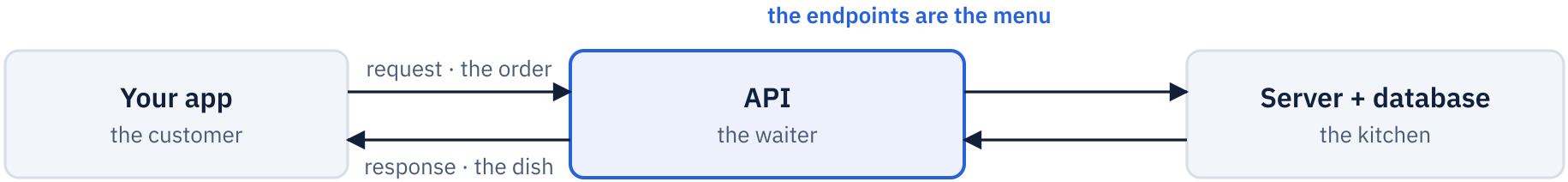
$0.00004

per question, by the repo's own estimate. Re-labelling and re-embedding a 394-row directory comes to about four cents.

Separate roles, by design

Cheap models handle structured jobs like classification; the capable one writes answers. Today's deployment points every role at Nova 2 Lite, but keeping the roles separate means each can be upgraded alone.

What an API is



You never walk into the kitchen yourself.

```
POST /api/v1/query HTTP/1.1
Authorization: Bearer eyJhbGciOi…
Content-Type: application/json

{ "question": "dentist that takes Medicaid",
  "member_address": "Dallas TX 75201" }
```

A request: method and path, headers, then a JSON body.

```
HTTP/1.1 200 OK
Content-Type: application/json

{ "message_id": "6f1c…",
  "primary_referrals":   [ { "name": "Smith Dental Clinic",
                             "phone": "(214) 555-0100" } ],
  "secondary_referrals": [ … ] }
```

A response: a status code, then JSON. Field names are illustrative.

200 OK · **202** accepted, not ready yet · **401** who are you? · **404** no such thing · **422** malformed request · **502** something behind me failed

FastAPI: endpoints from typed Python

```python
class QueryRequest(BaseModel):       # the shape of a valid request
    question: str
    page_limit: int = 5

@router.post("/query")               # POST /api/v1/query
async def query(req: QueryRequest,
                user=Depends(get_current_user)):   # auth first
    cards = run_search(req.question)
    return {"primary_referrals": cards}
```

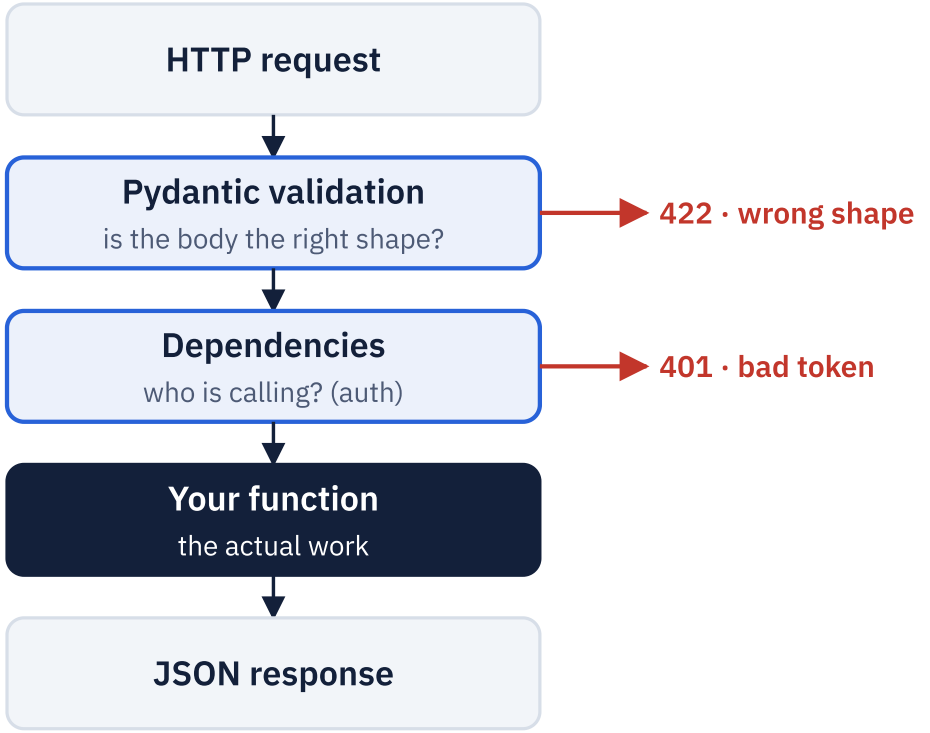
Simplified from `backend/api/routers/query.py`. Names are illustrative.

Docs for free

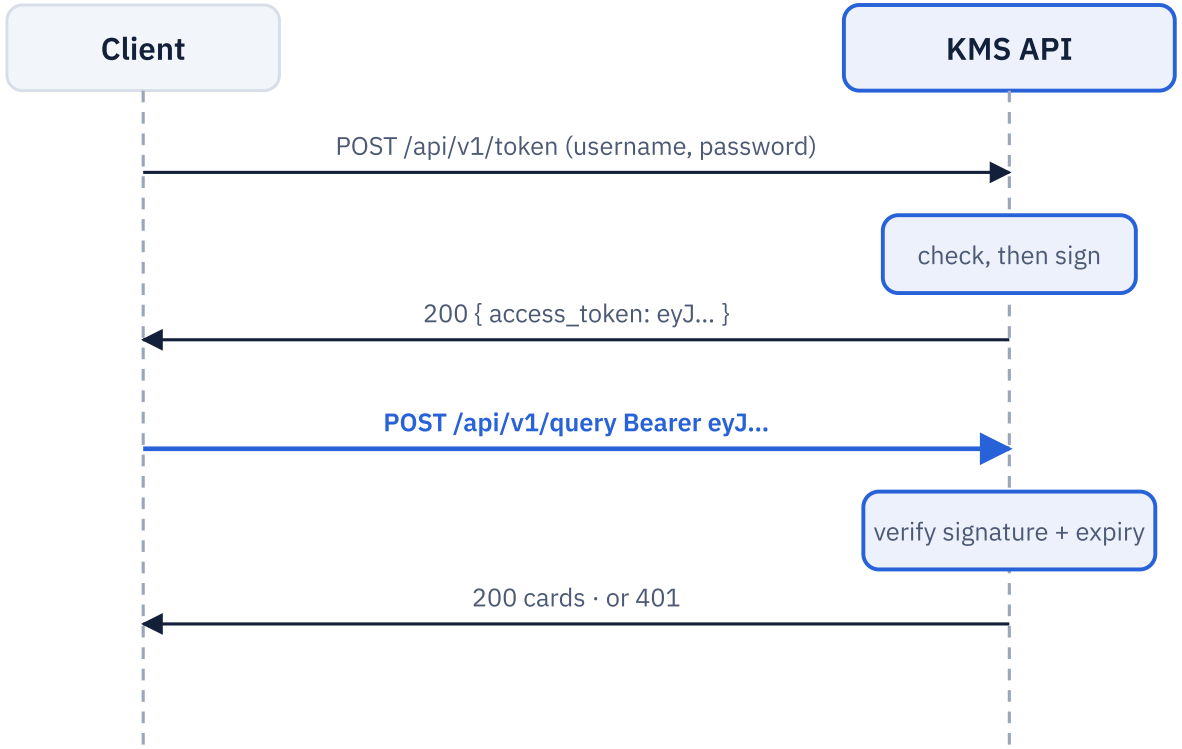
FastAPI reads the type hints and publishes interactive docs at `/api/docs`.

Lambda-ready

Mangum translates Lambda events into ordinary web requests, in `lambda_handler.py`.



Who is calling? Tokens and JWT



eyJhbGciOiJIUzI1NiJ9.eyJzdWIiOiJtbXItc2VydmljZSJ9.SflKxwRJSMeKKF2QT4

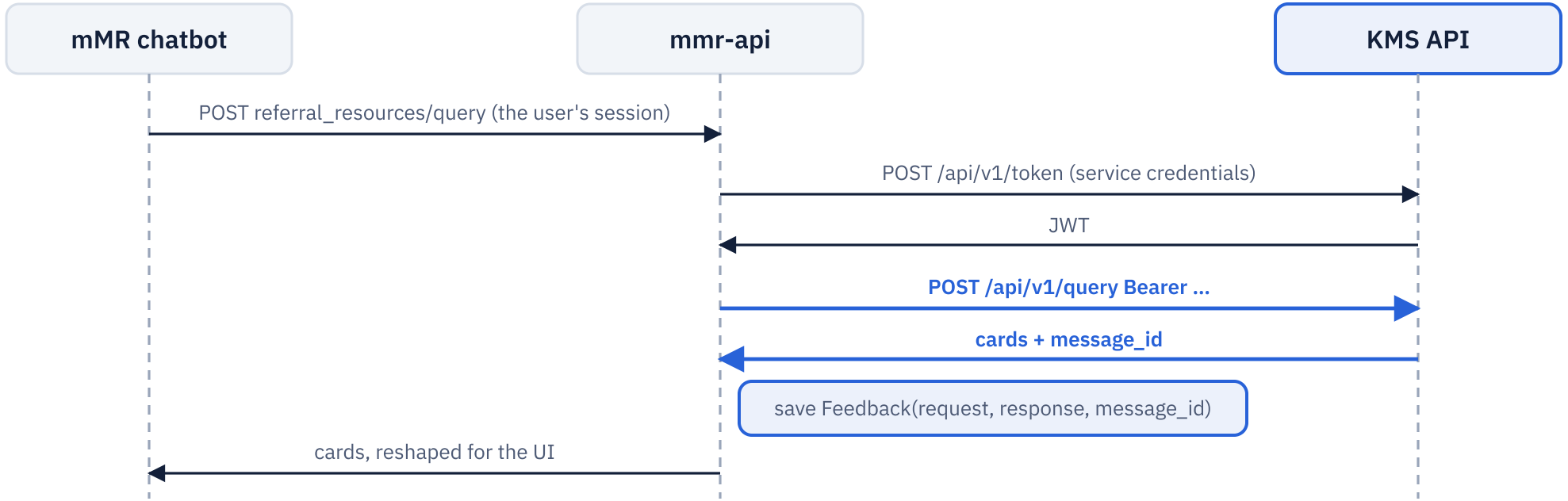
Header says how it's signed. **Payload** says who and until when. **Signature** proves the server issued it.

Anyone can decode the payload, so it never holds secrets. Only the server can make a valid signature, because only it has the signing key, kept in Secrets Manager.

> IN PLAIN WORDS
>
> A festival wristband. Show ID once at the gate, then flash the band at every stage without being checked again.

`/api/v2` serves the same handlers with hardened auth; v1 stays until it's retired.

mmr-api: the proxy in front of KMS



In mmr-api: `app/models/concerns/referral_resources/api.rb`

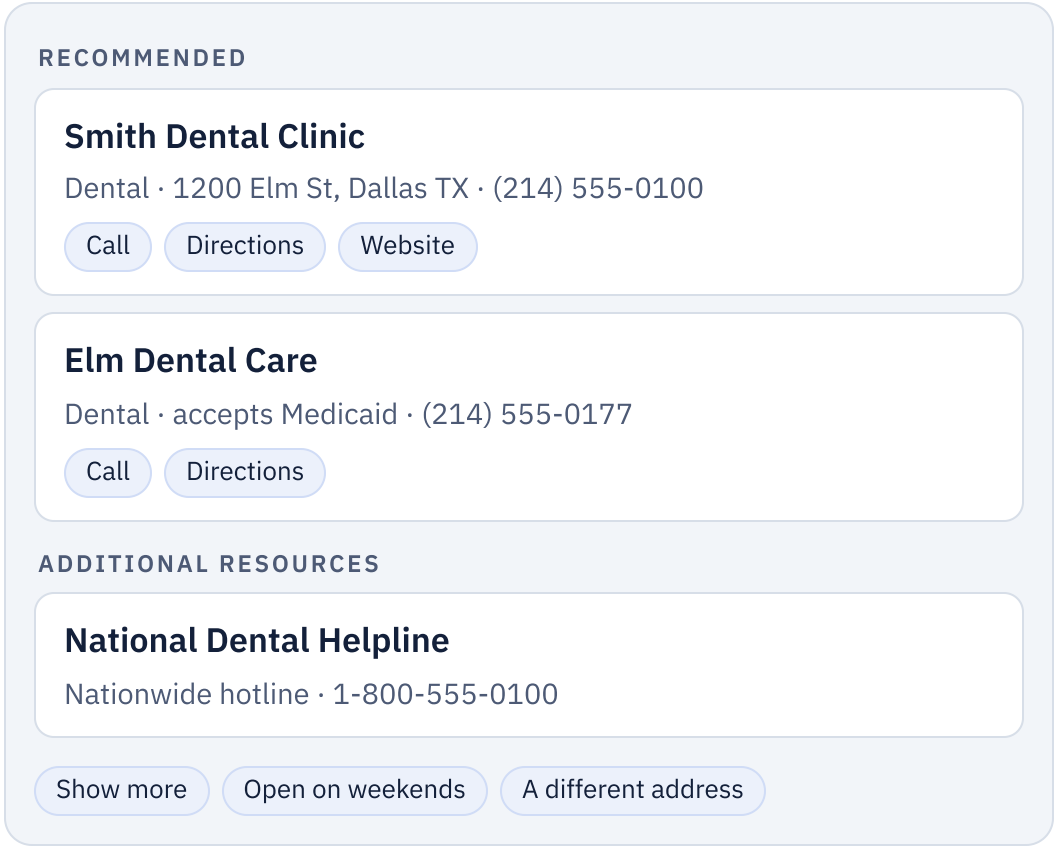
Keys stay server-side
No phone or browser ever holds a KMS credential.

The user's rules apply
mmr-api checks the signed-in user's permissions first.

Every answer is recorded
Saved with its message_id for rating and review.

Shaped for the screen
A serializer reshapes cards into what the app renders.

Referral cards, not paragraphs

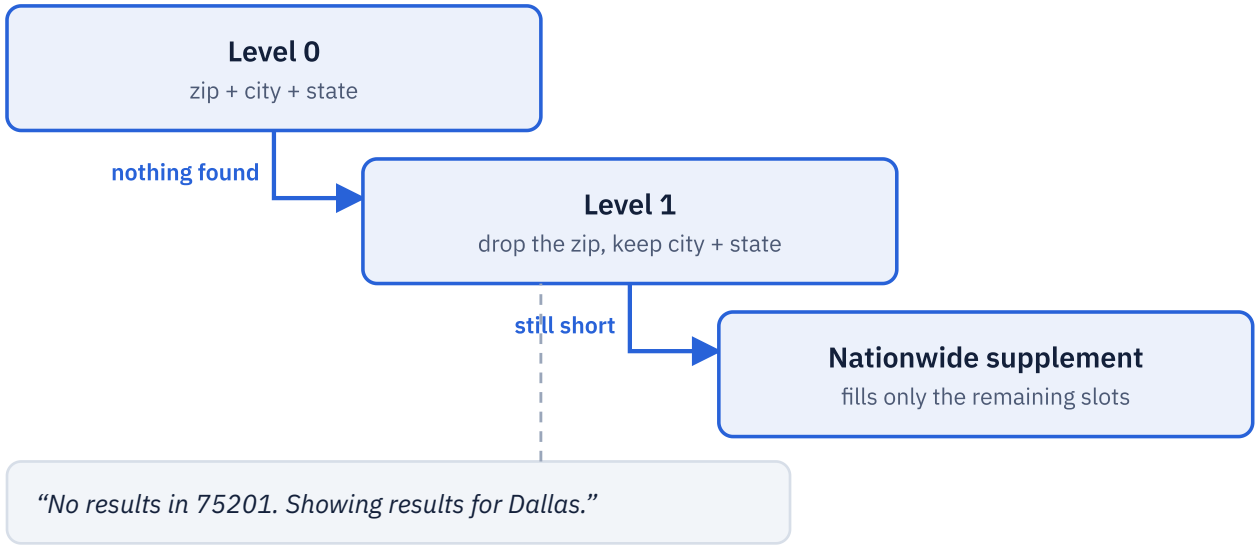


The app needs fields it can act on, a number to tap or an address to map, so KMS returns structured cards rather than prose.

- **Two sections.** Local resources first; national hotlines go under “additional resources”.
- **Composite dedup.** The same clinic listed in two directories appears once.
- **Session continuation.** “Any more?” plus the earlier `message_id` fetches the next page with no new model call.
- **Quick actions.** Suggestion buttons come back with every response, so the app shows them instantly.

Synthetic resources and illustrative suggestion labels.

When a search is too narrow: graceful fallback



An empty screen helps nobody, but neither does a national hotline ranked above a clinic down the street.

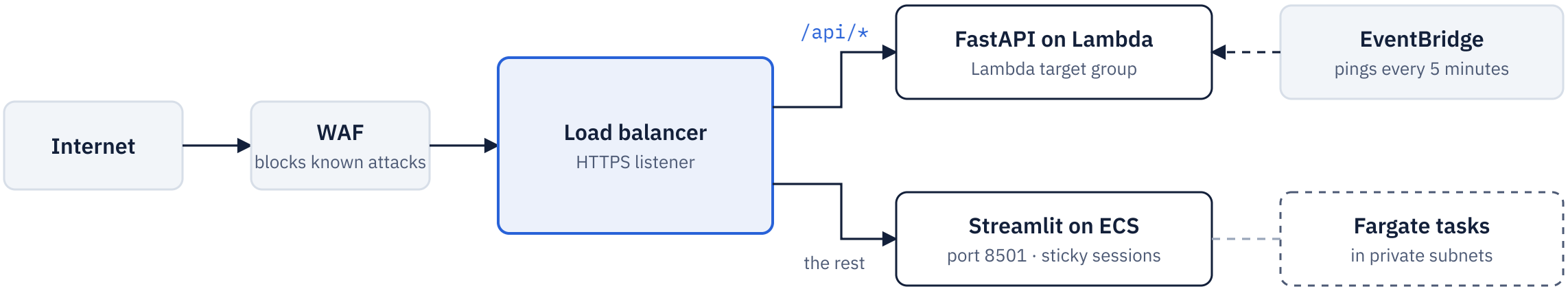
THE RULE IN THE CODE

National records are nationwide hotlines and portals with no local presence, so they must never outrank a real local resource. With local results present they go to the secondary section; only when there are none do they fill the main list.

IN THE CODE `backend/api/routers/query.py`

`_run_search_with_fallback()`

One front door, two back rooms



Lambda in one paragraph

Code that runs only when called, billed per request, with no server to manage. The catch is the cold start: the first call after a quiet spell waits while a container wakes.

Why the five-minute ping

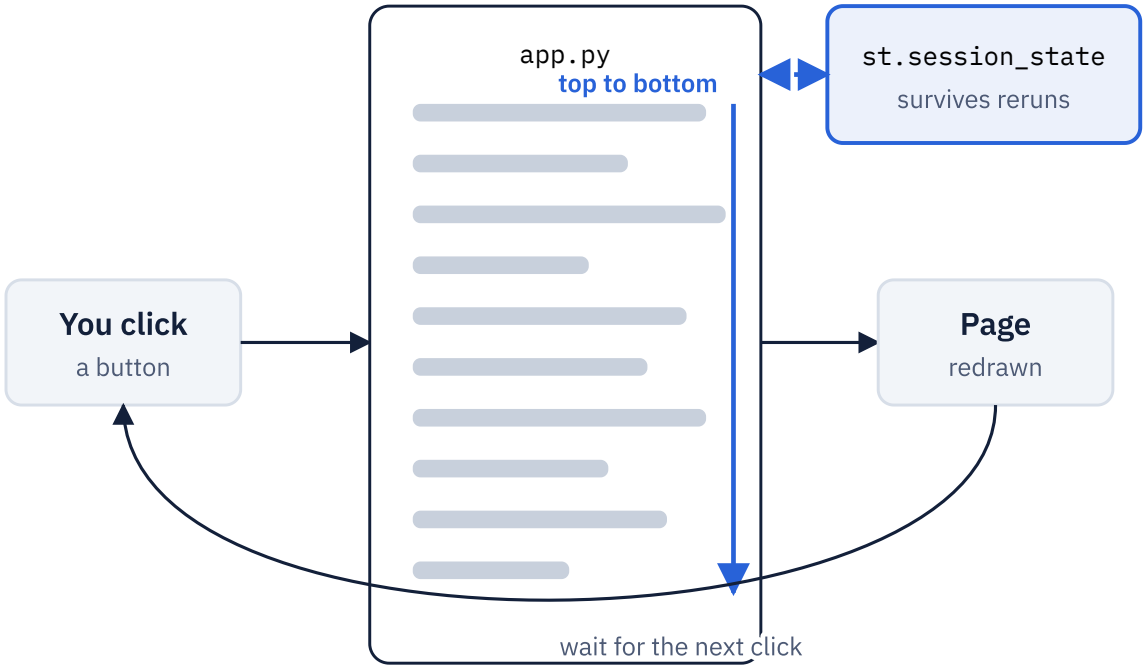
EventBridge calls the function on a schedule so a real request rarely meets a cold container. The handler recognises the ping and returns at once.

Why one balancer

One address, one certificate, one firewall in front of both apps. A path rule at priority 10 decides which room each request walks into.

IN THE CODE `infrastructure/terraform/lambda.tf` `aws_lb_listener_rule.api`

Streamlit: a web app that is just a Python script



There's no HTML or JavaScript to write. Every interaction reruns your script from the top, and Streamlit draws whatever it produces.

```
st.text_area("Session JSON", key="case_note_input")

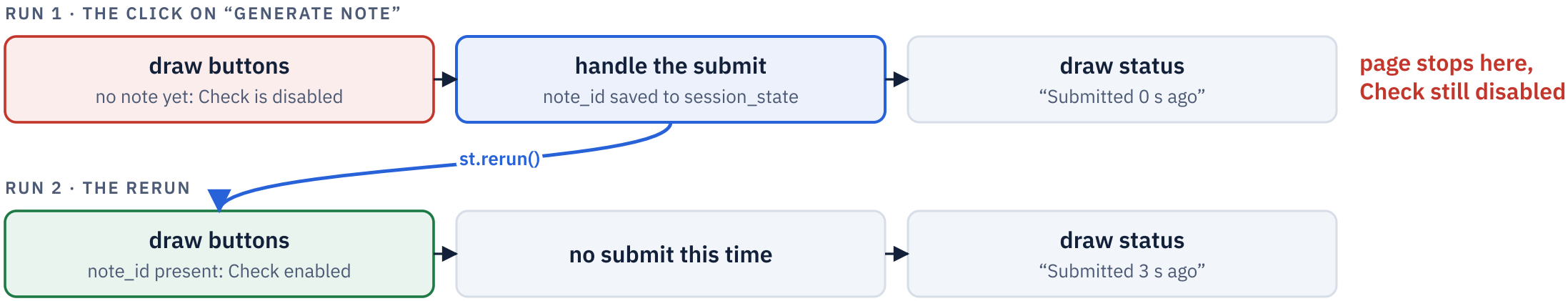
if st.button("Generate Note"):          # True only on the
    note_id = submit(payload)           # rerun after a click
    st.session_state["note_id"] = note_id
    st.rerun()                          # redraw with new state

st.write(st.session_state.get("note_id"))
```

IN PLAIN WORDS

Every click reprints the whole page. `session_state` is the sticky note that survives the reprint.

A real bug that teaches the model



The bug

After submitting, “Check for result” stayed greyed out. The buttons are drawn near the top; the submit handler runs further down. On the click's run the button was already drawn as disabled, and nothing redrew it.

The fix, and its catch

Call `st.rerun()` after a successful submit. But a rerun resets any widget without a key, so the input box also had to be keyed in session state, or the user's edits would vanish.

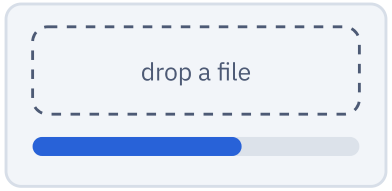
The admin console, page by page



Home

Status at a glance: documents, indexed rows, search-engine health. Also the login.

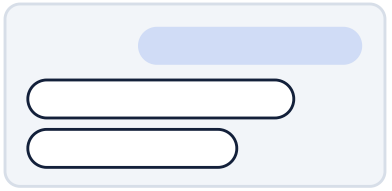
`Home.py`



Upload

Excel, PDF, Word or text in. A background job labels, embeds and indexes it.

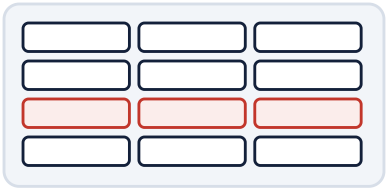
`pages/1_Upload.py`



Agentic Chat

Ask questions of the index and watch the search graph answer, with sources.

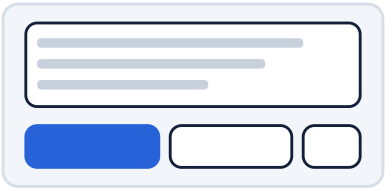
`pages/2_Agentic_Chat.py`



Data Viewer

Inspect each file's records and spot rows whose phone or e-mail disagrees with the provider's website.

`pages/3_Data_Viewer.py`



Case Note

Paste a call transcript and get a draft clinical note, brokered through mmr-api.

`pages/4_Case_Note.py`

IN PLAIN WORDS

The console is the workshop: where directories come in, where you check they arrived intact, and where you try the search before the app's users do.

What a DAP note is

After a call with a member, the care extender documents it in a standard clinical format. The service drafts it from the call's transcript.

```
[CARE_EXTENDER]: How has your sleep been this week?
[MEMBER]: Better. The wind-down alarm helps.
[CARE_EXTENDER]: Any trouble with your medication?
[MEMBER]: I missed Thursday but took it Friday.
```

Speaker-labelled transcript, plus the member's goals and referrals as JSON. Synthetic.

→

D **Data**
Reports improved sleep with a wind-down alarm; missed one dose Thursday, resumed Friday.

A **Assessment**
Progress on the sleep goal; the lapse was isolated and self-corrected.

P **Plan**
Reinforce the routine; review medication reminders at the next contact.

G **Goals**
Sleep goal in progress.

R **Referrals**
Behavioral health referral not discussed this session.

IN PLAIN WORDS

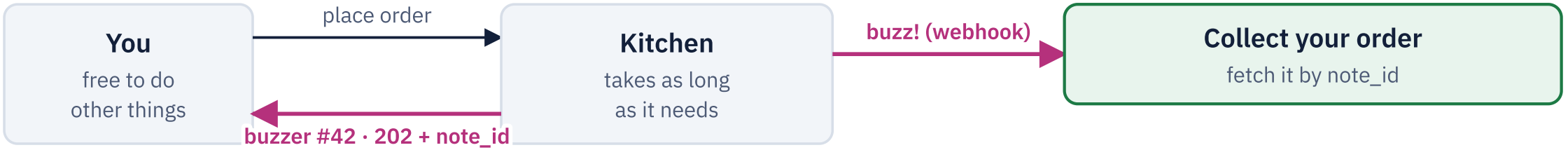
A first draft written from the conversation, so the care extender reviews and signs it rather than writing from a blank page.

Synchronous versus asynchronous

SYNCHRONOUS · THE COUNTER



ASYNCHRONOUS · THE BUZZER



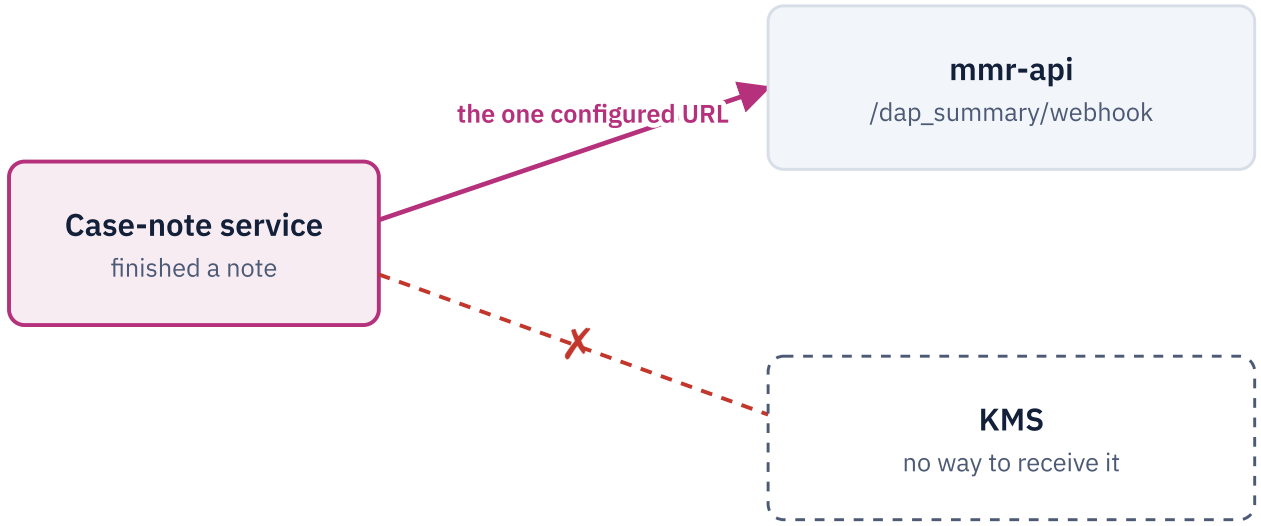
Why the service went async

Drafting a note can take longer than the roughly 29-second limit of the API gateway in front of the service. A synchronous call would simply be cut off.

What 202 means

“Accepted, not ready yet.” The response carries a `note_id`: the buzzer number you'll use to pick up the result.

A webhook delivers to one address only



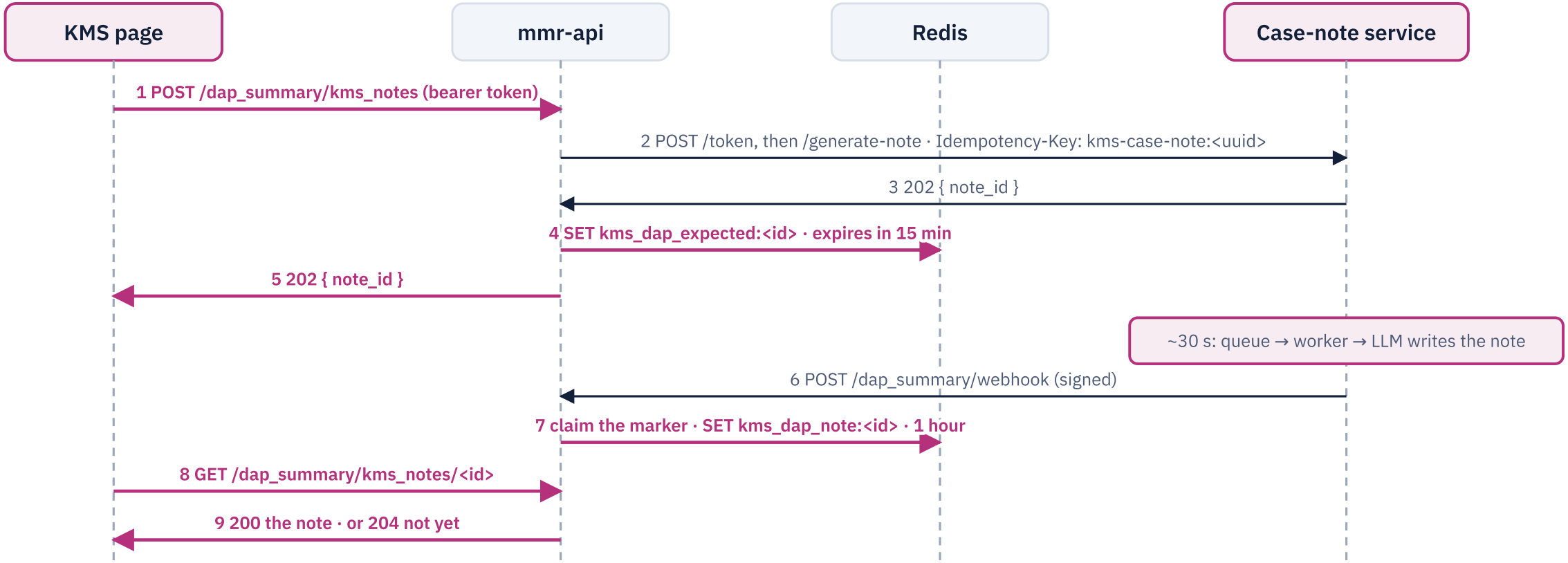
A webhook reverses the usual direction: instead of you asking “is it ready?”, the service calls *you* when it is. It calls exactly one URL, and that URL belongs to mmr-api.

- There is no “give me note 42” endpoint to poll.
- Changing the URL to KMS would steal every real note mmr-api is waiting for. An earlier attempt did exactly that and was reverted.
- So KMS has to collect its notes *through* mmr-api.

IN PLAIN WORDS

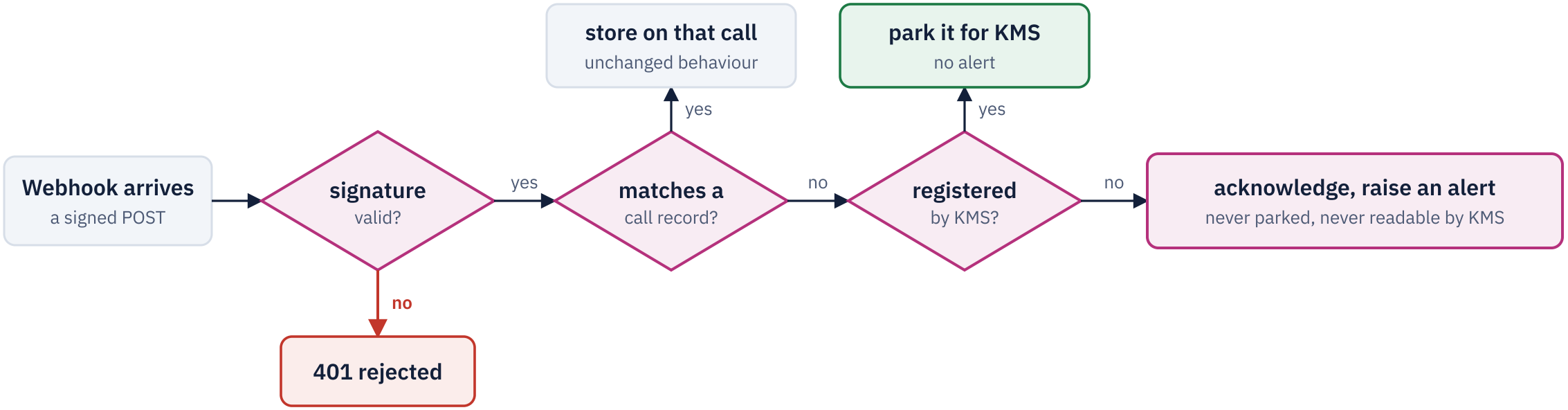
A restaurant that only delivers to the address on file. If you're at a friend's place, the food still goes home, and your friend has to hold it for you.

The broker, step by step



Steps 1 to 5 take under a second, and the page shows the note id. The user presses **Check for result**, which repeats step 8 until step 9 returns the note.

The safety boundary: only registered notes are parked



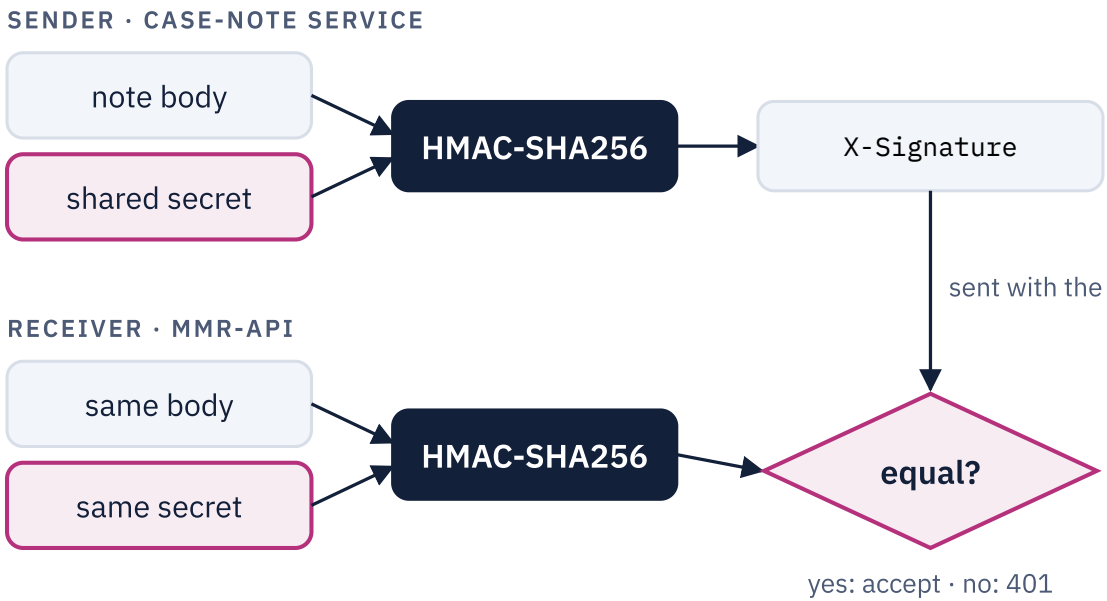
Why this matters

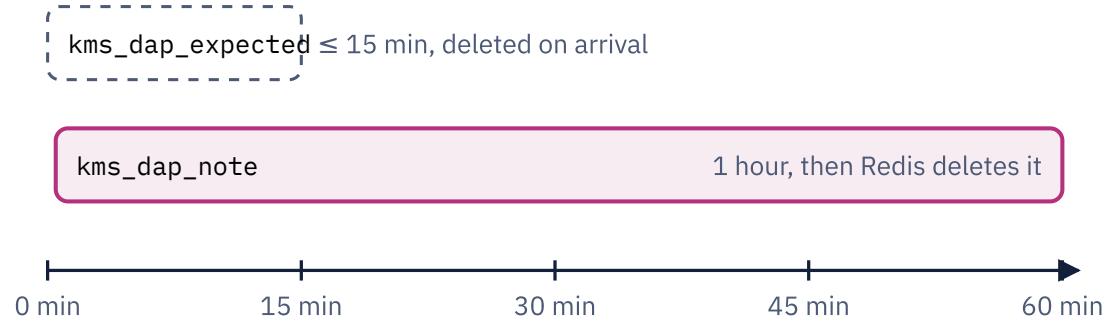
When a care extender regenerates a note, the superseded delivery also arrives matching no call. It's a real member's note. Nothing registered it, so it can never be read through the KMS route.

Why mmr-api mints the key

The service derives `note_id` from the idempotency key. If callers chose keys, one could name a real call's key and have that member's note parked. So mmr-api generates a fresh `kms-case-note:<uuid>` itself.

Seals and timers: HMAC signatures and expiring keys





Redis can attach an expiry to any key and deletes it itself when the time is up. Nothing has to clean up after the broker, and no clinical text lingers.

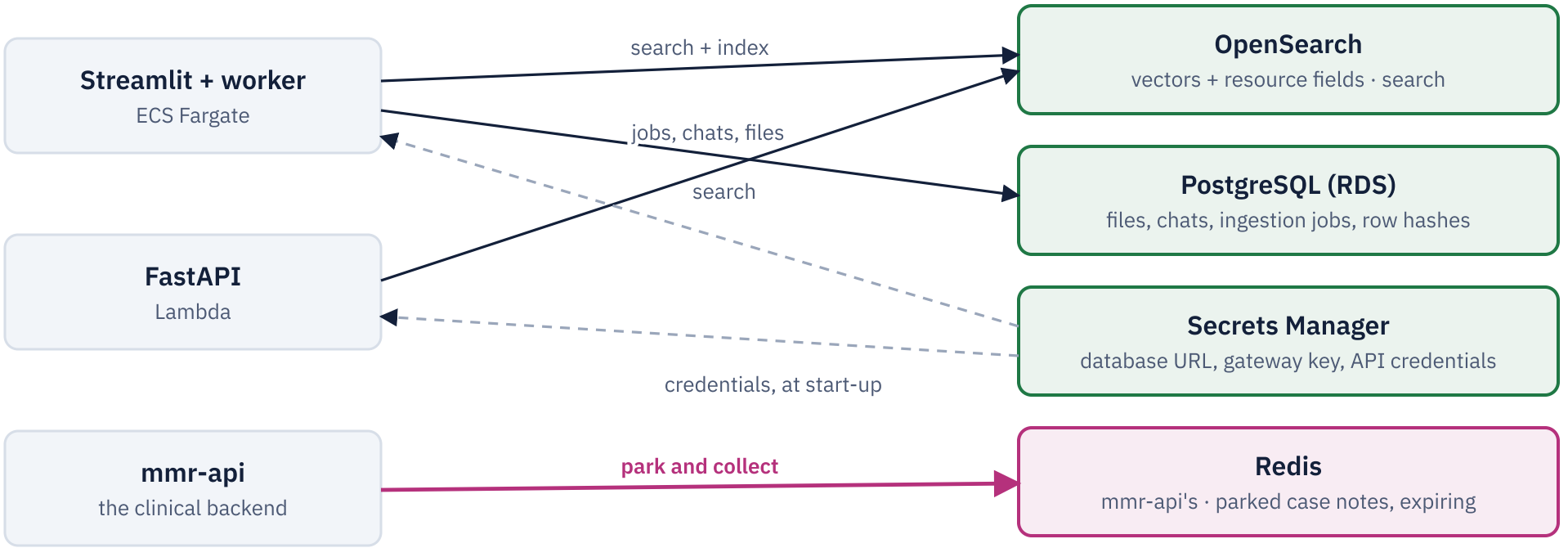
The comparison of signatures runs in constant time, so an attacker can't learn the right value from how long a wrong guess takes to fail.

IN PLAIN WORDS

A wax seal only the two of you can make. Alter one word of the letter and the seal no longer matches.

IN MMR-API `mmr-api · app/lib/kms_dap_parking.rb`

Who stores what



No line from FastAPI to PostgreSQL: the slim Lambda image has no database driver.

IN PLAIN WORDS

Each store is good at one thing: OpenSearch at finding similar things, PostgreSQL at keeping records straight, Redis at fast scratch space that expires, Secrets Manager at guarding keys.

Containers: the same box, everywhere

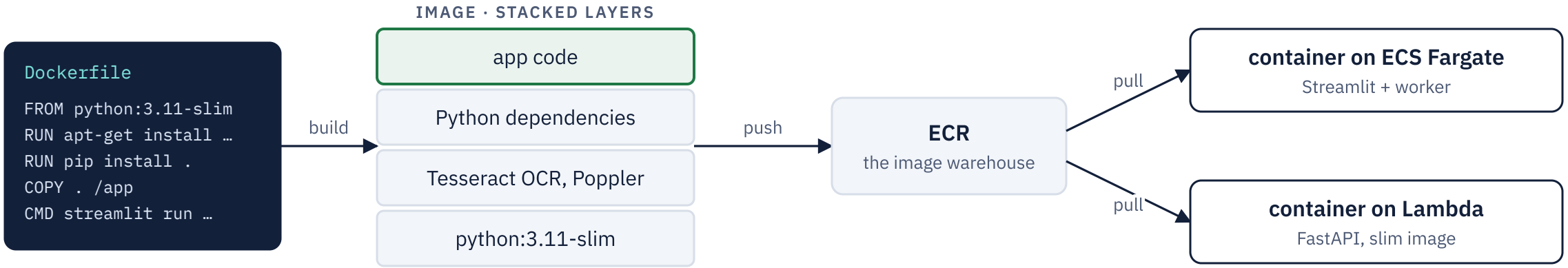


Image versus container

An **image** is the frozen recipe: operating system, Python, libraries and code. A **container** is a running copy of it. One image can run as many containers.

Why it matters

The exact image tested on a laptop is the one that runs in AWS. “Works on my machine” stops being an excuse.

Two images in KMS

The app image carries OCR tools for scanned PDFs. The Lambda image is deliberately slim, with no database driver, which is why it can't run the admin routes.

IN PLAIN WORDS

A shipping container: packed once, then carried by any ship, train or truck without being unpacked.

AWS building blocks, in plain words

VPC
Your private network in AWS. A gated neighbourhood.

Subnets
Streets inside it: public ones reachable from the internet, private ones not.

Load balancer (ALB)
The receptionist who sends each visitor to the right room.

WAF
The bouncer at the door, turning away known attack patterns.

ECS Fargate
Runs containers for you, with no servers to look after.

Lambda
Runs a function only when called; you pay per call.

ECR
The warehouse where container images are stored.

RDS
Managed PostgreSQL: backups and patching handled for you.

OpenSearch Service
Managed search and vector engine.

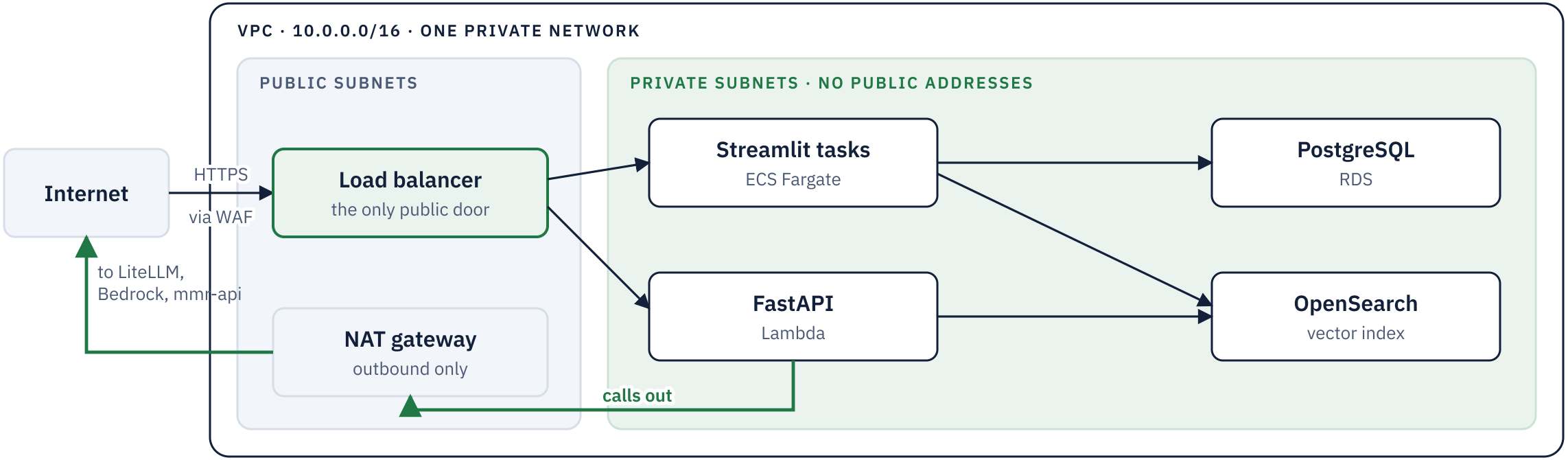
Secrets Manager
The safe for passwords and keys, with an audit trail.

NAT gateway
Lets private servers call out to the internet without being reachable from it.

CloudWatch + EventBridge
Logs, metrics and alarms; and a scheduler for timed jobs.

Every one of these appears in this project's Terraform. The next slide shows how they connect.

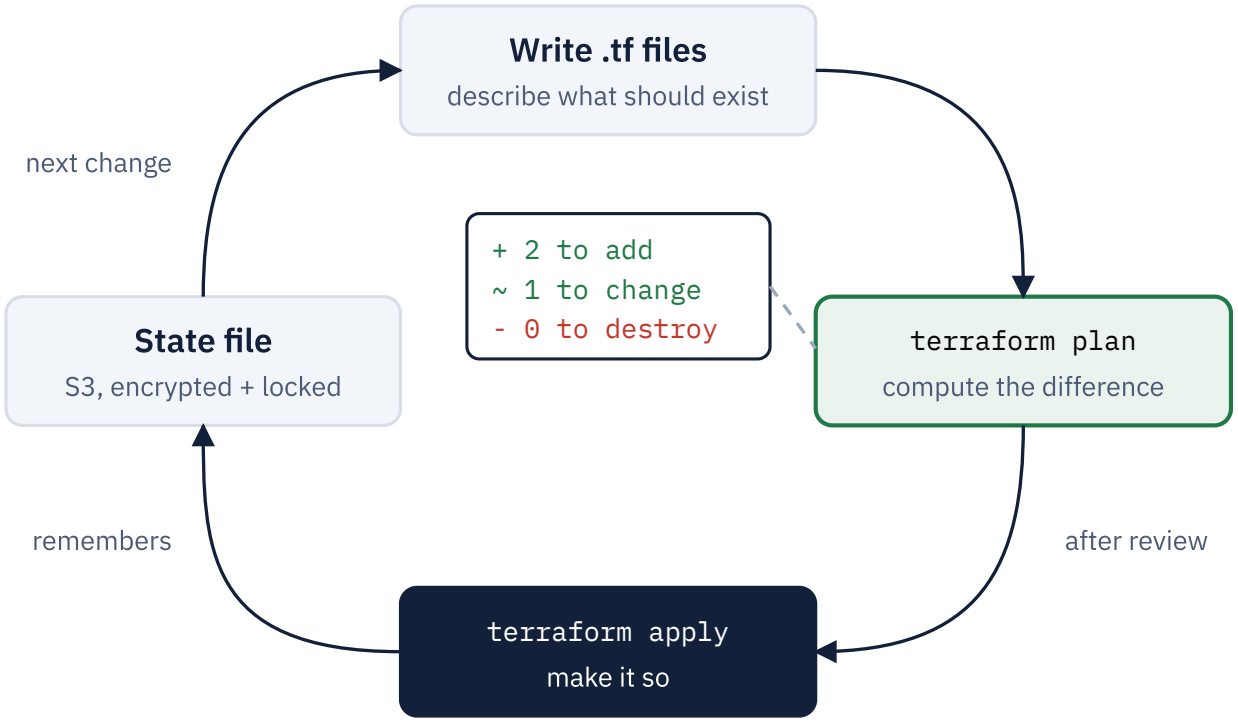
The network: what can reach what



CLOUDWATCH ALARMS WATCH

ECS CPU | ECS memory | load balancer 5xx | target 5xx | database connections | database storage | Lambda errors | Lambda duration

Terraform: infrastructure written as code



Instead of clicking through the AWS console, you describe the infrastructure you want in files. Terraform works out what to create, change or delete to make reality match.

```
resource "aws_ecs_service" "app" {
  name          = "${var.project_name}-service"
  desired_count = var.app_count
  launch_type   = "FARGATE"
}
```

IN PLAIN WORDS

A recipe instead of a meal. Anyone can rebuild the kitchen exactly, and every change is reviewed like code before it happens.

Terraform in this repo

`main.tf`	network, NAT, database, OpenSearch, load balancer, IAM
`ecs.tf`	cluster, task definition, service
`lambda.tf`	the function, `/api/*` routing, the warmer
`secrets.tf`	secret lookups and permission to read them
`alarms.tf`	CPU, memory, 5xx, database and Lambda alarms
`waf.tf`	firewall rules on the load balancer
`dns.tf`	domain records
`variables.tf · outputs.tf`	the inputs, and values reported back
`envs/sandbox.tfvars`	staging values
`envs/production.tfvars`	production values: same code

variable

A parameter of the code: “how many tasks?”, “which image?”

tfvars

The values for one environment. Sandbox and production share every .tf file and differ only here.

data source

A lookup of something that already exists, such as a secret's ARN, without Terraform owning it.



Secrets never touch the code



ECS: ARNs only

For the containers, Terraform never sees a value. The task definition names the secret; AWS injects it as the container starts.

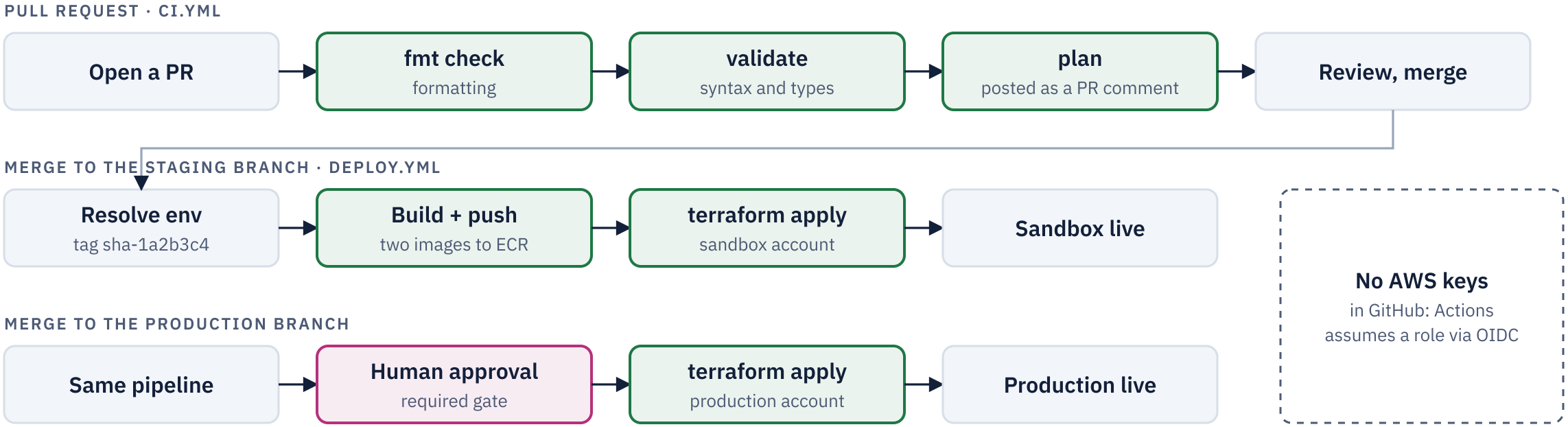
Lambda: an honest exception

The Lambda's keys are read by Terraform at deploy time and set as environment variables, so they pass through Terraform's state. That's why the state lives in an encrypted, locked bucket.

Fail closed

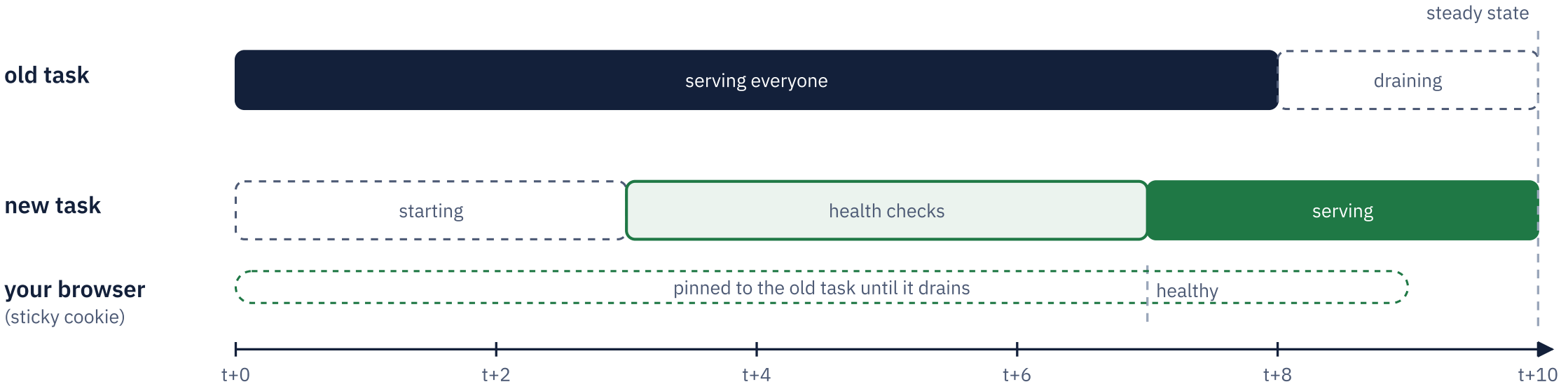
If a secret is missing, `terraform plan` fails and nothing deploys half-configured. A missing token on a route answers 401, never “let them in”.

From pull request to production



The image tag is the commit's short hash, so every running container can be traced back to the exact code it was built from.

Why “deployed” isn't “visible” yet



Green pipeline, old page

ECS starts the new task beside the old one and only retires the old one after the new one passes health checks and the old one drains, up to five minutes. Streamlit needs sticky sessions, so your own browser is the last to move.

How to know it's really done

The service's running count is back to its desired count, and the load balancer shows exactly one healthy target. A fresh private window then lands on the new code.

Building for healthcare

Minimum necessary

Hold clinical text only as long as needed. Parked notes expire in an hour; the case-note page never writes a note to a database.

Never log clinical content

Services log ids and statuses, never transcripts or note bodies. Logs travel further than databases.

Fail closed

Missing token: 401. Missing secret: the deploy stops. Unknown note: alert, don't expose.

Least privilege

Each role can read only the secrets it needs. The Lambda doesn't even ship a database driver.

Defence in depth

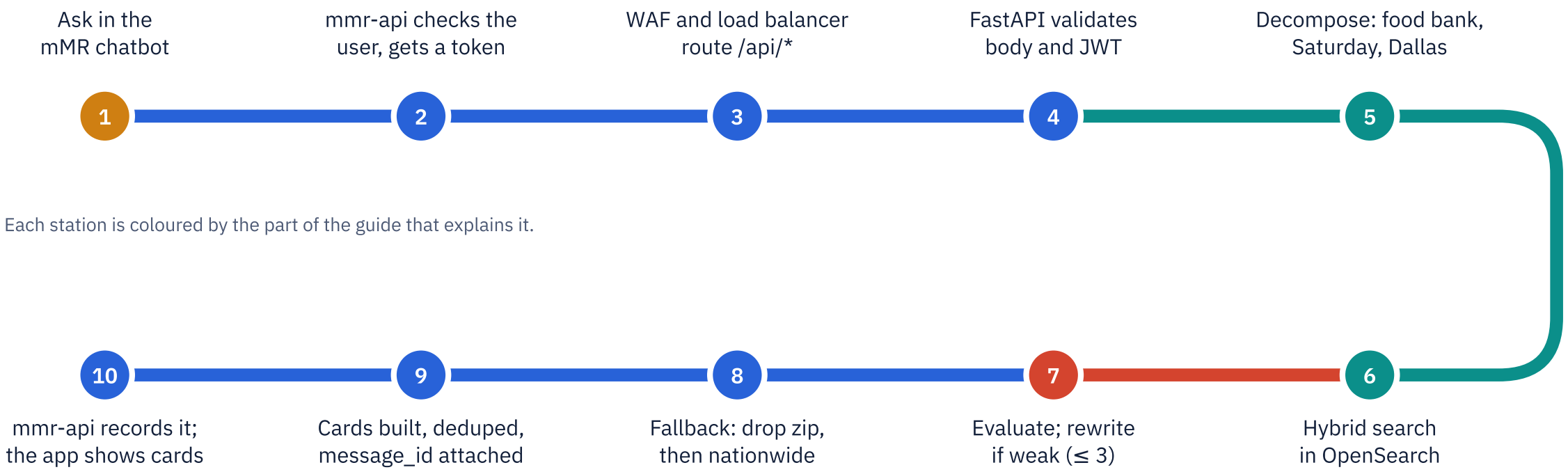
WAF, then the load balancer, then private subnets, then auth, then signature checks. No single lock.

Sanitise before sharing

No account ids, hostnames or real member data leave the company. This guide uses masked infrastructure and synthetic examples throughout.

One question, end to end

“A food bank near my member in Dallas that's open on Saturdays.” Here is everything that happens before the answer appears.



Each station is coloured by the part of the guide that explains it.

When things go wrong, and what the system does

Symptom	What happens	Where to look
Nothing matches the ZIP code	Drop the ZIP and say so, then top up with national hotlines below the local results.	`routers/query.py`
A phone number went stale	URL enrichment reads the provider's website; the Data Viewer flags rows where the two disagree.	`url_enricher.py` · Data Viewer
The first results are weak	Evaluate, rewrite the query and search again, at most three rounds.	`search_graph.py`
Note generation is slow	Async: a note id at once, a webhook later, the page shows how long it's been waiting.	the case-note broker
A token is wrong or missing	401, every time. Nothing proceeds on a guess.	`api/auth.py` · mmr-api's token check
Pipeline green, old page showing	Rolling deploy plus sticky sessions; wait for steady state.	ECS service · load balancer targets

Check yourself: AI and RAG

Q2 Why does KMS put a short context header in front of each chunk before embedding it?

Q3 A vector search for “dentist in Dallas” returns a perfect dental clinic in Houston. Which part of hybrid search prevents that?

Q4 What does a cosine similarity close to 1 tell you?

Q5 Why not simply fine-tune a model on the directories?

Q6 What stops the search graph from looping forever?

Q7 What is a token, and why does the context window matter for RAG?

Click a question to reveal the answer. All answers are also collected on the second-to-last slide.

Check yourself: apps and cloud

Q8 Why does the mMR app go through mmr-api instead of calling KMS directly?

Q9 Which requests does the load balancer send to Lambda, and which to ECS?

Q10 Why can't the case-note service deliver notes straight to KMS?

Q11 A webhook arrives for a note nobody registered. What happens?

Q12 What does `terraform plan` show, and why run it on every pull request?

Q13 The pipeline is green but you still see the old page. Why?

Try it yourself

EXERCISE 1

Run it locally

`docker compose -f docker/docker-compose.local.yml up --build -d`, open the console, upload a small spreadsheet of made-up resources and watch the ingestion job.

EXERCISE 2

Find the brake

In `search_graph.py`, find where `max_iterations` decides between reformulating and generating.

EXERCISE 3

Make it loop

In Agentic Chat, ask something vague, then something specific, and compare how many rounds each needs.

EXERCISE 4

Follow a secret

In `ecs.tf`, find where secrets become environment variables, then find the matching data source in `secrets.tf`.

EXERCISE 5

Rank national below local

In `routers/query.py`, find the comment that keeps national hotlines below local resources.

EXERCISE 6

Draw the broker

Sketch the nine broker steps from memory, then compare with the sequence diagram in part 6.

Use synthetic data only. Never copy real member information into a local setup.



A reading order for the code

1	`Home.py, pages/`	the console: the smallest surface
2	`utils/ingestion_graph.py`	how data gets in
3	`utils/contextual_chunker.py`	the context-header trick
4	`utils/search_graph.py`	the self-checking search
5	`utils/search_utils.py`	the hybrid query
6	`utils/query_processor.py`	decomposition and filters
7	`api/routers/query.py`	the production API: dense, read it last
8	`terraform/ecs.tf, workflows/deploy.yml`	how it ships

Then the docs

`docs/CODE_WALKTHROUGH.md`, `docs/FUNCTIONAL_DOCUMENTATION.md` and `docs/search-flow.md` go deeper than this guide on every station.

And the orientation file

`ORIENTATION.md` at the root of the shared folder is a one-page text version of this map.

Glossary

Agentic RAG
RAG that decides its own next step, such as searching again.

ALB
Application Load Balancer: routes requests by path.

API
A contract for programs to talk over HTTP.

Chunk
A small piece of a document, embedded on its own.

Cold start
The delay when a Lambda wakes after idling.

Container
A running copy of an image.

Context header
An LLM-written label put in front of a chunk.

Context window
How much text a model can consider at once.

Cosine similarity
Similarity as the angle between two vectors.

DAP note
Data, Assessment, Plan: a clinical note format.

Embedding
Text turned into a vector of numbers by meaning.

Fine-tuning
Further training of a model on new examples.

Hallucination
A confident, made-up answer.

HMAC
A signature proving a message is untouched.

HNSW
A layered graph for fast nearest-neighbour search.

Hybrid search
Vector, keyword and filters in one query.

Idempotency key
An id that makes a retried request safe.

JWT
A signed token that says who you are.

Lambda
Functions that run on demand, billed per call.

LangGraph
A library for AI workflows as graphs.

LiteLLM
A gateway giving every model one interface.

OpenSearch
The search engine holding the vector index.

Pydantic
Validates data against Python type hints.

RAG
Retrieve facts, then generate from them.

Terraform
Infrastructure described as code.

Token
A word-piece, the unit models read and write.

TTL
Time to live: when a stored key expires.

Webhook
A service calling you back when work is done.



Answers

Q1 What does RAG stand for?
Retrieval-augmented generation: look the facts up first, then let the model write an answer from them. Part 3 is all about it.

Q2 Why does KMS put a short context header in front of each chunk before embedding it?
A bare row like “Smith Dental | 555-0100” carries little meaning. The header says what and where it is, so its embedding lands near the right questions. Anthropic's benchmark reports about 49% fewer failed retrievals.

Q3 A vector search for “dentist in Dallas” returns a perfect dental clinic in Houston. Which part of hybrid search prevents that?
The term filters. City and state are hard rules that no similarity score can override.

Q4 What does a cosine similarity close to 1 tell you?
The two embeddings point in almost the same direction: very similar meaning, whatever their length.

Q5 Why not simply fine-tune a model on the directories?
It would need retraining whenever a number changed, couldn't cite sources, and could still invent details. RAG reads current records at question time.

Q6 What stops the search graph from looping forever?
`max_iterations = 3`. After three rounds it answers from the best results it has.

Q7 What is a token, and why does the context window matter for RAG?
A word-piece. The window caps how much text the model can consider at once, so retrieval has to choose the few most relevant chunks.

Q8 Why does the mMR app go through mmr-api instead of calling KMS directly?
It keeps KMS credentials server-side, applies the user's permissions, records each exchange, and shapes the data for the screen.

Q9 Which requests does the load balancer send to Lambda, and which to ECS?
Paths starting `/api/` go to the FastAPI Lambda; everything else goes to Streamlit on ECS.

Q10 Why can't the case-note service deliver notes straight to KMS?
Its webhook goes to one configured URL, mmr-api's. Pointing it at KMS would steal mmr-api's real deliveries.

Q11 A webhook arrives for a note nobody registered. What happens?
mmr-api acknowledges it and raises an alert, but never parks it, so it's never readable through the KMS route.

Q12 What does `terraform plan` show, and why run it on every pull request?
The difference between the code and real infrastructure. Reviewers see the infrastructure effect before merging, and a failing plan blocks the PR.

Q13 The pipeline is green but you still see the old page. Why?
A rolling deploy: the old task drains while the new one passes health checks, and the sticky cookie keeps your browser on the old one. Wait until the running count equals the desired count.



You can now explain KMS end to end

You've followed every line: why the platform exists, the AI underneath it, how answers stay grounded in real records, the apps and the broker, and how it all ships.

- Run it locally with synthetic data and break it on purpose.
- Read the code in the order on the reading-order slide.
- Come back to any slide: each one stands on its own.

Prepared as a sanitised guide: masked infrastructure identifiers, synthetic data throughout.

